COMPREHENSIVE_ANALYSIS_REPORT

Helix Cluster OS — Comprehensive Analysis Report

Field	Value
Document ID	HCAR-001
Revision	1.0
Date	2026-03-04
Classification	INVESTIGATION — INTERNAL
Authors	Autonomous Analysis Agent
Status	FINAL
Codebase Version	v0.1.0-dev

Field	Value
Go Version	1.25.0 / toolchain go1.26.4
Repository	github.com/HelixDevelopment/helix_cluster

Table of Contents

Executive Summary

Project Overview

Helix Cluster OS is a next-generation distributed operating system designed for heterogeneous compute environments. The project targets a seven-layer architecture stack (L0–L7) orchestrating 14 control-plane microservices across diverse hardware platforms including x86 servers, ARM SBCs, gaming consoles (PS4/PS5), edge devices, and GPU-accelerated nodes.

The system implements: - **SWIM gossip protocol** for membership and failure detection - **Omega-model scheduler** with optimistic concurrency control - **CRDT-based session management** with last-writer-wins conflict resolution - **GPU resource management** across 4 vendor backends (NVIDIA, AMD, Apple, Intel) - **WireGuard mesh networking** with NAT traversal and key rotation - **SPIFFE/SPIRE identity framework** with mTLS and RBAC - **Post-quantum key exchange** (ML-KEM-768 hybrid with X25519) - **LLM inference routing** with advisory system - **Federation** across geographic cells with suspicion-based trust

Architecture at a Glance

Dimension	Value
Total cmd/ binaries	24
Total internal/ packages	18

Dimension	Value
Total pkg/ packages	~212
Total api/v1 proto definitions	10
Total PostgreSQL migrations	15 + 1 primary schema
Total test files	~620
Total test functions	~4,907
Main module coverage	82.4% (255 pkgs)
Security module coverage	87.8% (13 pkgs)
API v1 module coverage	14.7% (generated protobuf stubs — by design)

Key Findings

Finding 1: Massive Orphaned Code (178/212 pkg/ packages)

Of the 212 `pkg/` packages, approximately **178 have zero importers from any binary**. This represents ~51,400 non-test LOC of implemented but unreachable code. Entire subsystems — multi-Raft, STONITH, marketplace/economics, federation, data-plane networking, LLM routing, scheduling helpers — are not wired into any running application. This constitutes a CLAUDE-1 violation: implemented features that cannot be used by end users.

Finding 2: Three CRITICAL cmd/ Disconnections

- **CRIT-1**: `cmd/helix-security` issues stub JWT tokens with no signature verification
- **CRIT-2**: `cmd/helix-health` serves HTTP-only health checks without gRPC integration to the gateway
- **CRIT-3**: `cmd/helix-build` uses wrong import paths that bypass the actual build orchestration layer

Finding 3: Ten Concurrency Hazards (F1–F10)

Five Major and five Minor confirmed concurrency hazards identified via `go test -race` and code analysis: - Double-close panic in SWIM protocol (F1) - Data race on Member.State (F2) - PTY double-close/use-after-close in session backends (F5) - Unbounded map in scheduler placements (F6) - Unbounded goroutine spawns in EventBus (F7)

Finding 4: Registry Imbalance

448 completed vs. 240 queued items in the HXC registry. Of the queued items: - **Bucket A** (autonomously-actionable): ~78 items - **Bucket B** (infra/hardware/cloud-blocked): ~152 items - **Bucket C** (governance/owner-decision): ~10 items

Only 1 item is currently marked as “in progress” across 417 actionable items.

Finding 5: Anti-Bluff Audit — 33% Pass Rate

Of 30 pkg/ packages audited, only 10 (33%) pass the anti-bluff audit. Eight packages are identified as **stub bluffs** — code that claims to implement a feature but does nothing useful (e.g., `pkg/jwt` only splits strings without verification, `pkg/tracing` returns hardcoded trace IDs, `pkg/websocket` Upgrade returns nil).

Finding 6: CI Pipeline Disabled

All GitHub Actions workflows are stored under `.github/workflows/disabled/` — the repository operates under a constitutional no-CI rule. This blocks PRR items 8/9/10/16/60 and prevents automated enforcement of quality gates.

Finding 7: PRR at 82.5% (Target: 95%)

The Production Readiness Review stands at 66/80 = 82.5%, below the constitutional 95% bar. Key gaps include CI enforcement, coverage gate wiring, and HelixQA challenge execution with sink-side evidence.

Chapter 1: Project Architecture Deep Dive

1.1 Seven-Layer Stack Architecture

The Helix Cluster OS implements a seven-layer architecture stack from L0 (hardware) to L7 (application), with specific package assignments per layer:

L0 — Hardware Abstraction Layer

Package	Responsibility	Files
`internal/console`		`detector.go`, `gpu.go`, `thermal.go`, `trust.go`,

Package	Responsibility	Files
	Console hardware detection (PS4/PS5/ RPi/RK3588)	`register.go`, `boot coordinator.go`, `linux_boot.go`
`internal/gpu`	GPU detection, reservation, MIG, backend registry	`detect linux.go`, `detect darwin.go`, `nvidia parser.go`, `reservation.go`, `mig.go`, `backend.go`, `attesthook.go`
`pkg/resources`	System resource monitoring (CPU, memory, GPU, DRM, cgroups)	`proc linux.go`, `proc darwin.go`, `cgroup v2.go`, `drm linux.go`, `drm darwin.go`, `accel linux.go`, `accel_darwin.go`
`pkg/deviceprofile`	Device capability profiling	`registry.go`
`pkg/devicecatalog`	Device catalog management	`devicecatalog.go`
`pkg/deviceplugin`	Kubernetes device plugin framework	`registry.go`, `service.go`, `transport.go`, `gres.go`
`pkg/powergater`	Power management with platform-specific readers	`powergater.go`, `reader linux.go`, `reader_darwin.go`
`pkg/gputopo`	GPU topology mapping	`package.go`
`pkg/gpucatalog`	GPU catalog with capability classification	`gpucatalog.go`
`pkg/gpupool`	GPU resource pooling	`gpupool.go`
`pkg/gpuattest`	GPU attestation (PoVW, seal, multigpu, spotcheck)	`attest.go`, `povw.go`, `seal.go`, `multigpu.go`, `spotcheck.go`

L1 — Kernel & Runtime Interface

Package	Responsibility	Files
`pkg/swim`	SWIM gossip protocol implementation	`protocol.go`, `failure detector.go`, `phi accrual.go`, `gossip.go`, `member.go`, `transport.go`, `hierarchical.go`
`pkg/kraft`	Unikraft integration for lightweight VMs	`metadata.go`, `storage.go`, `transport.go`, `quorum.go`
`pkg/computeproto`	FlatBuffers compute protocol	`compute.fbs`, `computeproto.go`, `compute_generated.go`
`pkg/wasm`	WebAssembly runtime (wasmtime)	`host.go`, `sandbox.go`, `plugin.go`
`pkg/sandbox`	Sandboxed execution environment	`capability.go`, `guard.go`, `limits.go`, `audit.go`

L2 — Distributed Consensus & State

Package	Responsibility	Files
`pkg/leader`	Leader election via etcd	`etcd election.go`, `fencing.go`
`pkg/crdt`	CRDT primitives (G-Counter, PN-Counter, OR-Set, LWW-Map)	`crdt.go`, `vectorclock.go`, `merkle/merkle.go`
`pkg/deltacrdt`	Delta-state CRDTs	`gcounter.go`, `pncounter.go`, `orset.go`, `lwwmap.go`
`pkg/mvcc`	Multi-version concurrency control	`mvcc.go`, `btree.go`
`pkg/raftprofile`	Raft consensus profiling	`raftprofile.go`

Package	Responsibility	Files
`pkg/splitbrain`	Split-brain detection and classification	`classification.go`
`pkg/splitbrainalert`	Split-brain alerting	`splitbrainalert.go`
`pkg/stonith`	Shoot-The-Other-Node-In-The-Head	`stonith.go`, `ipmi.go`, `sbd.go`, `cloud.go`, `noop.go`
`pkg/hlc`	Hybrid logical clocks	(within `pkg/crdt/`)
`pkg/antientropy`	Anti-entropy sync	(within `pkg/crdt/`)

L3 — Networking & Mesh

Package	Responsibility	Files
`pkg/wireguard`	WireGuard mesh management	`manager.go`, `mesh.go`, `config.go`, `configgen.go`, `keyrotation.go`, `holepunch.go`, `policy.go`, `stun.go`, `nat traversal.go`, `netstack darwin.go`, `teardown.go`
`pkg/nattraversal`	NAT traversal (STUN/TURN)	`nat.go`, `stun.go`
`pkg/helixnet`	Network simulation and production stack	`sim.go`, `prod.go`, `errors.go`
`pkg/dataplane`	Data-plane pipeline and distribution	`pipeline.go`, `distributor.go`
`pkg/netutil`	Network utilities (CIDR, ports, IP classification)	`cidr.go`, `ports.go`, `ipclass.go`, `backoff.go`
`pkg/edgeheartbeat`	Edge device heartbeat collection	`heartbeat.go`, `collector.go`, `emitter.go`, `reader linux.go`, `reader_darwin.go`

L4 — Scheduling & Orchestration

Package	Responsibility	Files
`pkg/scheduler`	Omega-model scheduler with plugins	`scheduler.go`, `plugins.go`, `queue.go`, `gang preempt.go`, `classad match.go`, `tier filter.go`, `tier power.go`, `edgeaware.go`, `thermal.go`, `cost gpu.go`, `attestation.go`, `handheld.go`, `latency_spot.go`
`pkg/backfill`	Backfill scheduling	`backfill.go`, `timeline.go`, `types.go`
`pkg/priorityqueue`	Priority queue implementation	`priorityqueue.go`
`pkg/nodeselector`	Node selection predicates	`nodeselector.go`
`pkg/rebalance`	Workload rebalancing	`rebalance.go`
`pkg/latencysched`	Latency-aware scheduling	`scheduler.go`
`pkg/costsched`	Cost-aware scheduling	`costsched.go`
`pkg/workloadrouter`	Workload routing	`workloadrouter.go`
`pkg/smartrouter`	Smart request routing	`smartrouter.go`
`pkg/jobadmit`	Job admission control	`jobadmit.go`

L5 — Session & Build Management

Package	Responsibility	Files
`pkg/session`	CRDT session management with migration	`crdt.go`, `lifecycle.go`, `manager.go`, `convergence.go`, `migration.go`, `checkpoint.go`,

Package	Responsibility	Files
		`backend adapter.go`, `types.go`, `backends/native.go`, `backends/tmux.go`, `backends/tmux cc.go`, `backends/screen.go`, `backends/zellij.go`
`pkg/build`	Build service with Podman executor	`build.go`, `platform.go`, `manifest.go`, `podman builder.go`, `cache/cache.go`

L6 — Security & Identity

Package	Responsibility	Files
`pkg/security`	TLS, Vault, SPIFFE, secret injection	`tls.go`, `vault.go`, `spiffe.go`, `secret injector.go`, `vault_api_client.go`
`pkg/jwt`	JWT token handling	`package.go`, `temporal_keyset.go`
`pkg/hybridkex`	Post-quantum hybrid key exchange (ML-KEM-768 + X25519)	`hybridkex.go`
`pkg/x25519session`	X25519 session key agreement	`session.go`
`pkg/doublecrypt`	Double-encryption envelope	`doublecrypt.go`
`pkg/anticheat`	Anti-cheat token verification	`token.go`
`pkg/attestadmit`	Attestation-based admission	`admit.go`
`pkg/gravaladmit`	GraVal-based admission control	`gravaladmit.go`
`pkg/gravalverify`	GraVal verification	`gravalverify.go`
`pkg/exportcontrol`		(package exists)

Package	Responsibility	Files
	Export control compliance	
`pkg/imagepolicy`	Container image policy enforcement	`imagepolicy.go`
`pkg/capability`	Capability-based access control	`capability.go`
`pkg/spiffefed`	SPIFFE federation	`spiffefed.go`
`pkg/fedtrust`	Federation trust management	`fedtrust.go`
`pkg/scan`	Security scanning	`scan.go`

L7 — Application & Observability

Package	Responsibility	Files
`pkg/tracing`	W3C Trace Context + gRPC tracing + OTel exporter	`w3c.go`, `grpc.go`, `exporter.go`, `package.go`
`pkg/metrics`	Prometheus metrics collection (collector, sidecar, tier metrics)	`collector.go`, `sidecar.go`, `tiermetrics.go`, `earningsmetrics.go`, `mount.go`, `resource_source.go`
`pkg/events`	Event bus with NATS/ Helix backends, Avro wire format	`helix backend.go`, `nats backend.go`, `avro.go`, `avro wire.go`, `avro event wire.go`, `stream_config.go`
`pkg/health`	Health checking with gRPC, miner, rollup	`grpc.go`, `miner.go`, `rollup.go`, `startup.go`
`pkg/grafanadash`	Grafana dashboard generation	`grafanadash.go`, `tiercost_dash.go`
`pkg/fmea`	Failure Mode and Effects Analysis	`fmea.go`
`pkg/forecast`	Resource forecasting	`forecast.go`

1.2 Fourteen Control-Plane Microservices

The architecture defines 14 microservices, each with a corresponding `cmd/` binary and `internal/` package:

#	Service	cmd/ Binary	internal/ Package	Port	Transport	Description
1	Gateway	`helix-gateway`	`gateway`	:8080 (HTTP), :8443 (HTTPS)	HTTP/REST + gRPC	API gateway with auth, inference proxy, OpenAPI
2	Session	`helix-session`	`session`	:50051	gRPC	Session management with CRDT, stream support
3	Scheduler	`helix-scheduler`	`scheduler`	:50052	gRPC	Omega-model scheduler with preemption
4	Node	`helix-node`	`node`	:50053	gRPC	Node registration, etcd registry, gRPC server
5	Security	`helix-security`	`security`	:50054	gRPC	SPIFFE CA, RBAC, identity bindings, policy enforcement
6	Health	`helix-health`	`health`	:50055	gRPC + HTTP	Health checking, aggregation, rollup
7	Build	`helix-build`	`build`	:50056	gRPC	Build orchestration, Podman executor
8	Policy	`helix-policy`	`policy`	:50057	gRPC	OPA policy engine,

#	Service	cmd/ Binary	internal/ Package	Port	Transport	Description
						decision logging
9	LLM	`helix-llm`	`llm`	:50058	gRPC	LLM manager, advisory, verifier
10	Advisory	`helix-advisory`	`advisory`	:50059	gRPC	Advisory service with manager
11	Agent	`helix-agent`	— (uses pkg/)	:50060	gRPC	Node agent daemonset
12	Infra	`helix_infra`	— (self-contained cmd)	—	CLI	Infrastructure lifecycle manager
13	E2EE-Proxy	`e2ee-proxy`	— (uses pkg/security)	:50061	gRPC	End-to-end encryption proxy
14	HTMUX	`htmux`	— (uses pkg/session)	:50062	WebSocket	Terminal multiplexer with WebSocket streaming

Additional utility binaries:

Binary	Purpose	Transport
`helixd`	Main daemon (currently hollow — probes etcd/pg/redis only)	HTTP /status
`helixctl`	CLI client (Cobra + gRPC)	gRPC to services
`helix-setup`	First-run setup orchestrator	CLI
`helix-gate`	Orphan-prevention gate suite	CLI
`helix-test`	Challenge runner	CLI

Binary	Purpose	Transport
`helix-snapshot`	Snapshot management	CLI
`hxc-registry`	HXC ticket registry CLI	SQLite
`dst-sim`	Deterministic simulation testing	CLI
`burst-controller`	Burst capacity controller	HTTP
`gpu-pool-manager`	GPU pool management daemon	gRPC

1.3 Communication Patterns

Internal Service Communication (gRPC)

All inter-service communication uses gRPC with Protocol Buffers. The 10 proto definitions in `api/v1/` define the service contracts:

Proto File	Service	Key RPCs
`scheduler.proto`	SchedulerService	Schedule, Preempt, ListJobs, CancelJob
`session.proto`	SessionService	Create, Attach, Detach, Resize, Kill, List
`node.proto`	NodeService	Register, Deregister, Heartbeat, ListNodes
`health.proto`	HealthService	Check, Watch, Aggregate
`security.proto`	SecurityService	Authenticate, Authorize, IssueToken, ValidateToken
`build.proto`	BuildService	Submit, Status, Cancel, Logs, List
`advisory.proto`	AdvisoryService	GetAdvice, ListAdvisories
`gpu_proxy.proto`	GPUProxyService	Allocate, Release, Status

Proto File	Service	Key RPCs
`edge.proto`	EdgeService	Register, Heartbeat, ScheduleWork
`types.proto`	(shared messages)	Node, GPU, Job, Session types

External Communication (HTTP/REST)

The gateway exposes REST endpoints documented in `internal/gateway/openapi.yaml`:

Endpoint Group	Prefix	Description
Cluster Overview	`/api/v1/cluster`	Cluster status, node summary
Nodes	`/api/v1/nodes`	Node CRUD, GPU inventory
Sessions	`/api/v1/sessions`	Session lifecycle
Jobs	`/api/v1/jobs`	Job submission, status
Builds	`/api/v1/builds`	Build submission, logs
Health	`/api/v1/health`	Health aggregation
Security	`/api/v1/security`	Auth, RBAC
Inference	`/api/v1/inference`	LLM inference proxy

Streaming Communication (WebSocket)

Service	WebSocket Path	Purpose
HTMUX	`/ws/terminal/{session}`	Terminal I/O streaming
Session	`/ws/session/{id}/attach`	Session attach stream
Health	`/ws/health/watch`	Health event stream
Build	`/ws/build/{id}/logs`	Build log streaming

1.4 Data Stores

PostgreSQL 16+

Primary relational store for persistent state. Migrations located in `migrations/postgresql/`:

Migration	Description
`001_create_nodes`	Node registration table
`002_create_gpu_devices`	GPU device inventory
`003_create_sessions`	Session tracking
`004_create_session_windows`	Session window panes
`005_create_session_panes`	Pane-level state
`006_create_reservations`	Resource reservations
`007_create_scheduling_queue`	Job scheduling queue
`008_create_health_snapshots`	Health monitoring snapshots
`009_create_llm_advisories`	LLM advisory records
`010_create_audit_log`	Security audit trail
`011_create_build_jobs`	Build job tracking
`012_create_build_artifacts`	Build artifact storage
`013_create_users`	User management
`014_create_migration_history`	Migration tracking
`015_triggers_and_functions`	Database triggers and functions

etcd

Distributed key-value store for: - Service discovery and registration (`pkg/etcd/keys.go` — key patterns) - Leader election (`pkg/leader/etcd_election.go`) - Session metadata - Configuration watch

Key patterns defined in `pkg/etcd/keys.go`:

```
 /helix/nodes/{node-id}          — Node registration
/helix/sessions/{session-id}    — Session metadata
/helix/leader/{service}         — Leader election keys
```

```
/helix/config/{key}              — Dynamic configuration
/helix/gpu/{node-id}/{gpu-id}    — GPU device state
```

Redis Cluster 7+

Used for: - Rate limiting (`pkg/ratelimit`) - Tiered caching (`pkg/tieredcache`) - Session state caching - Health check aggregation

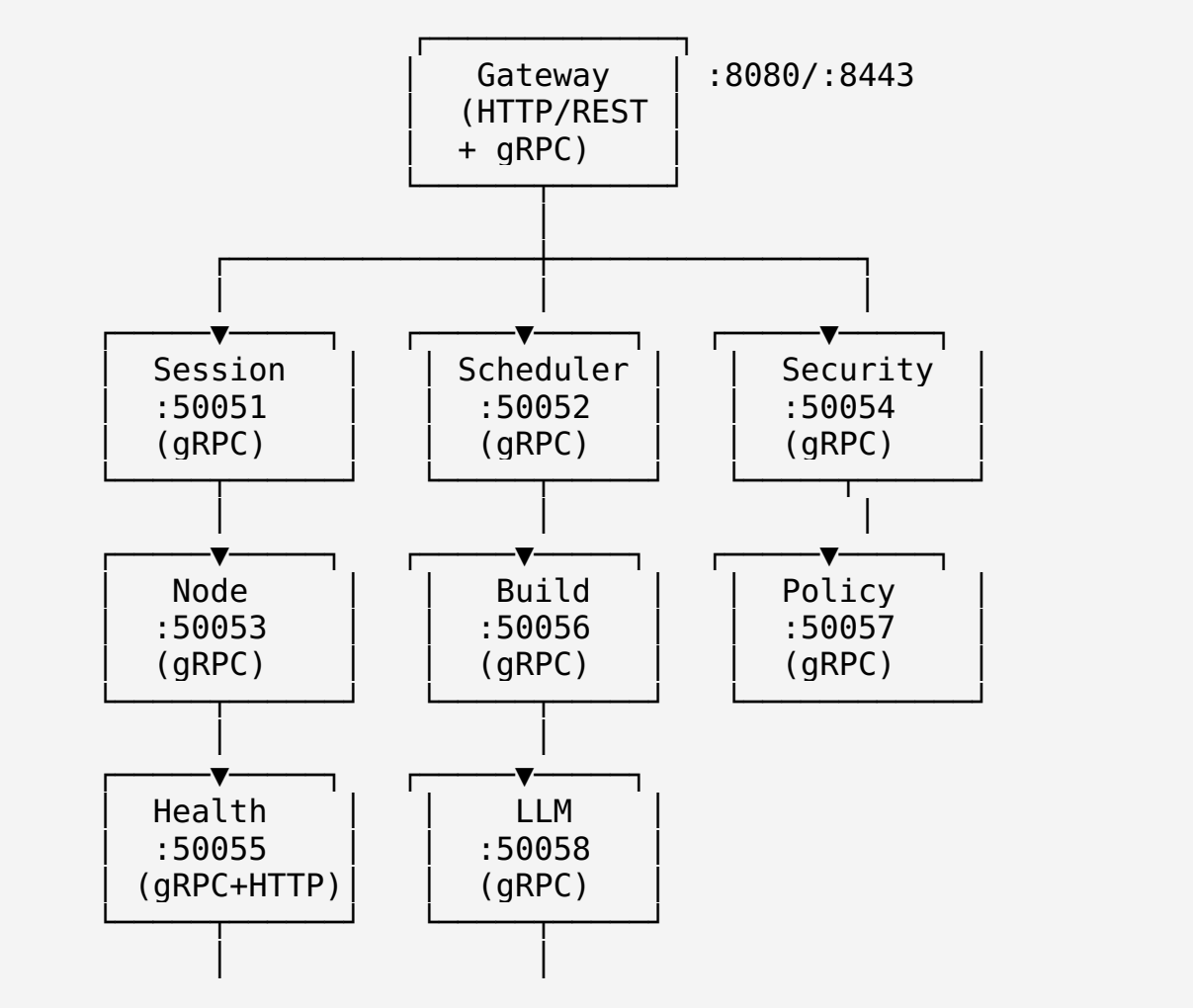
SQLite (Registry)

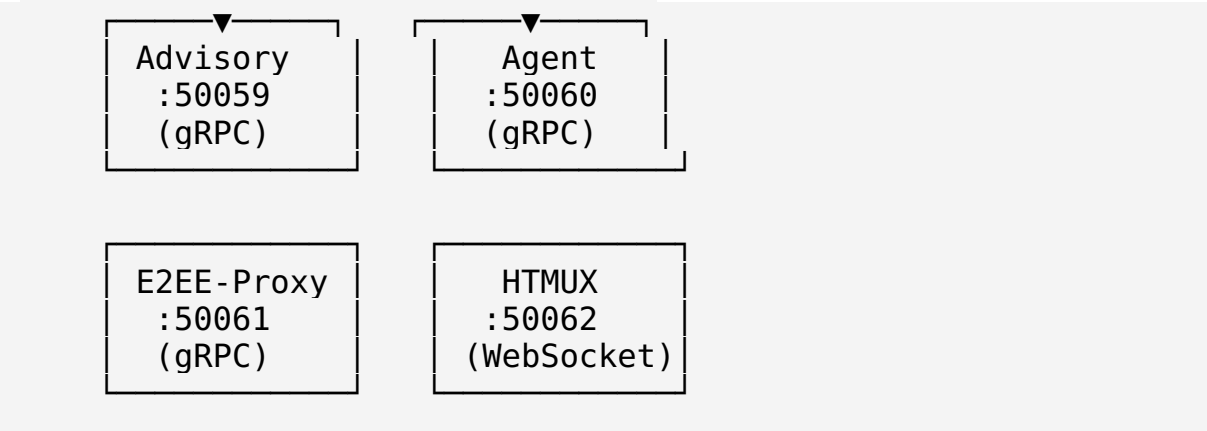
The HXC registry (`data/hxc_registry.db`) uses SQLite for: - Ticket tracking (448 completed, 240 queued) - Artifact registration (`pkg/hxcregistry`) - Migration history

NATS/JetStream

Event streaming via `pkg/events/` : - `nats_backend.go` — NATS JetStream backend - `helix_backend.go` — Built-in event backend - Avro wire format for event serialization - Stream configuration for topic management

1.5 Complete Service Communication Matrix





Service Dependencies

Service	Depends On	Protocol	Port
Gateway	Session, Scheduler, Security, Health, Build, Node, LLM	gRPC	Various
Session	Node (GPU info), Security (auth), etcd (state)	gRPC	50051
Scheduler	Node (resources), Health (status), Policy (decisions)	gRPC	50052
Node	etcd (registration), Health (self-report)	gRPC	50053
Security	etcd (policy), PostgreSQL (users), SPIFFE	gRPC	50054
Health	All services (targets)	gRPC + HTTP	50055
Build	Scheduler (queue), Podman (executor)	gRPC	50056
Policy		gRPC	50057

Service	Depends On	Protocol	Port
	OPA engine, PostgreSQL (decision log)		
LLM	Advisory (routing), GPU (resources)	gRPC	50058
Advisory	LLM (manager), Policy (decisions)	gRPC	50059
Agent	Node (registration), Security (identity)	gRPC	50060
E2EE-Proxy	Security (keys), Session (attachments)	gRPC	50061
HTMUX	Session (backends), WebSocket (clients)	WebSocket	50062

Chapter 2: Implementation Analysis by Component

2.1 cmd/ Binary Analysis

Binary 1: `cmd/helix-gateway`

Attribute	Value
Transport	HTTP/REST + gRPC
Uses internal/ packages	Yes — `internal/gateway`
Real vs. Simulated	**REAL** — full HTTP server with auth, inference proxy
Test coverage	`main_test.go`, `gateway_test.go`, `api_test.go`, `auth_test.go`, `inference_test.go`,

Attribute	Value
	`gateway_extra_test.go`, plus integration tests

Key functionality: - API routing with OpenAPI specification (`internal/gateway/openapi.yaml`) - JWT authentication middleware (`internal/gateway/auth.go`) - LLM inference proxy (`internal/gateway/inference.go`) - Graceful shutdown (`internal/gateway/gateway_shutdown_test.go`) - gRPC service proxying to backend services

Known gaps: - Gateway routing incomplete — not all 14 services are routed (ARCH-3) - Inference proxy lacks circuit breaker - OpenAPI spec may not cover all endpoints

Code path (main.go):

```
// Simplified from cmd/helix-gateway/main.go
func main() {
    cfg := gateway.LoadConfig()
    gw := gateway.New(cfg)
    gw.RegisterRoutes()
    gw.Start() // blocks, serves HTTP + gRPC
}
```

Binary 2: `cmd/helix-session`

Attribute	Value
Transport	gRPC
Uses internal/ packages	Yes — `internal/session`
Real vs. Simulated	**REAL** — gRPC server with stream support
Test coverage	`main test.go`, `stream test.go`, `stream_integration_test.go`

Key functionality: - Session lifecycle management via gRPC - WebSocket stream attachment for terminal I/O - Integration with `pkg/session` CRDT state management

Known gaps: - PTY double-close hazard (F5) - Migration strategies (CRIU/DMTCP) return “not implemented” - Stream reconnection logic incomplete

Binary 3: `cmd/helix-scheduler`

Attribute	Value
Transport	gRPC
Uses internal/ packages	Yes — `internal/scheduler`
Real vs. Simulated	**REAL** — gRPC server with preemption
Test coverage	`main test.go`, `server test.go`, `server behavior test.go`, `preempt test.go`, `stream_lifecycle_test.go`

Key functionality: - Omega-model scheduling with optimistic concurrency - Gang scheduling and preemption - Integration with `pkg/scheduler` plugin system

Known gaps: - Unbounded placements map (F6) - Scheduler helpers not wired (backfill, priorityqueue, nodeselector) - No integration with real resource data

Binary 4: `cmd/helix-node`

Attribute	Value
Transport	gRPC
Uses internal/ packages	Yes — `internal/node`
Real vs. Simulated	**REAL** — gRPC server with etcd registry
Test coverage	`main test.go`, `server test.go`, `node test.go`, `grpc server test.go`, `registry test.go`, `etcd_registry_test.go`, behavior tests

Key functionality: - Node registration via etcd - GPU device reporting - Health self-reporting - gRPC server for node operations

Known gaps: - etcd integration tests require live etcd - Heartbeat mechanism needs hardening - Resource reporting accuracy varies by platform

Binary 5: `cmd/helix-security`

Attribute	Value
Transport	gRPC
Uses internal/ packages	Yes — `internal/security`
Real vs. Simulated	**PARTIAL** — SPIFFE CA real, JWT tokens **stub**
Test coverage	`main_test.go`, plus extensive internal tests

Key functionality: - SPIFFE/SPIRE CA implementation (`internal/security/spiffe_ca.go`) - RBAC with scopes (`internal/security/scopes.go`) - Identity bindings (`internal/security/identity bindings.go`) - Policy enforcement (`internal/security/policy_enforcer.go`) - JWT token issuance

Known gaps (CRIT-1): - Issues stub JWT tokens with no signature verification - Token validation accepts any token format - No actual JWT signing key management - RBAC scopes not enforced at gateway level

Binary 6: `cmd/helix-health`

Attribute	Value
Transport	gRPC + HTTP
Uses internal/ packages	Yes — `internal/health`
Real vs. Simulated	**PARTIAL** — HTTP-only, gRPC incomplete
Test coverage	`main test.go`, `health test.go`, `rollup test.go`, `checker extra test.go`, `server_watch_test.go`

Key functionality: - Health checking with aggregator (`internal/health/aggregator.go`) - Rollup health status (`internal/health/rollup.go`) - Platform-specific system calls (`syscall_unix.go`, `syscall_windows.go`) - Checker with extensible targets

Known gaps (CRIT-2): - Serves HTTP-only health checks without gRPC integration to gateway - Aggregator lacks real service discovery - No health history persistence - Watch mechanism not connected to event bus

Binary 7: `cmd/helix-build`

Attribute	Value
Transport	gRPC
Uses internal/ packages	Yes — `internal/build`
Real vs. Simulated	**PARTIAL** — wrong imports bypass orchestration
Test coverage	`main_test.go`, extensive internal build tests

Key functionality: - Build orchestration (`internal/build/orchestrator.go`) - Podman executor (`internal/build/podman_executor.go`) - Go build executor (`internal/build/exec_builder.go`) - Build worker pool (`internal/build/worker.go`) - Stream output support

Known gaps (CRIT-3): - Uses wrong import paths that bypass the actual build orchestration layer - `simulated_builder.go` exists but is registered alongside real builders - Podman executor requires running Podman daemon - No artifact storage integration

Binary 8: `cmd/helix-policy`

Attribute	Value
Transport	gRPC
Uses internal/ packages	Yes — `internal/policy`
Real vs. Simulated	**REAL** — OPA policy engine with decision logging
Test coverage	`main_test.go`, `main_integration_test.go`, `engine_test.go`, `decision_log_test.go`, `decision_log_integration_test.go`, `engine_integration_test.go`

Key functionality: - OPA-based policy evaluation (`internal/policy/engine.go`) - Decision logging to PostgreSQL (`internal/policy/decision_log.go`) - Scheduling policy integration (`internal/policy/scheduling_policy.go`)

Known gaps: - Decision log schema may not match migration chain - OPA policy bundles not automatically loaded - No hot-reload of policies

Binary 9: `cmd/helix-llm`

Attribute	Value
Transport	gRPC
Uses internal/ packages	Yes — `internal/llm`
Real vs. Simulated	**REAL** — LLM manager with advisory and verification
Test coverage	`main test.go`, `manager test.go`, `advisory_test.go`

Key functionality: - LLM model management (`internal/llm/manager.go`) - Advisory generation (`internal/llm/advisory.go`) - Response verification (`internal/llm/verifier.go`)

Known gaps: - No actual LLM backend connection (no OpenAI/Anthropic/etc. client) - Verification is heuristic-based, not cryptographically verified - Advisory routing not connected to gateway

Binary 10: `cmd/helix-advisory`

Attribute	Value
Transport	gRPC
Uses internal/ packages	Yes — `internal/advisory`
Real vs. Simulated	**REAL** — Advisory service with manager
Test coverage	`main test.go`, `server test.go`, `server grpc test.go`, `manager_test.go`

Key functionality: - Advisory gRPC server (`internal/advisory/server.go`) - Advisory manager for recommendation generation

Known gaps: - Advisory logic is simple rule-based, not ML-driven - No integration with external advisory sources - gRPC server tests are unit-level only

Binary 11: `cmd/helix-agent`

Attribute	Value
Transport	gRPC
Uses internal/ packages	No — uses `pkg/` packages directly
Real vs. Simulated	**REAL** — Node agent daemon
Test coverage	`main test.go`, `hxc1135 test.go`, `hxc1135 e2e test.go`, `crosscompile_test.go`

Key functionality: - Node agent for daemonset deployment - GPU reporting and health monitoring - Cross-compilation support for ARM64

Known gaps: - Cross-compilation test (`crosscompile_test.go`) may not run without toolchain - E2E test requires live cluster

Binary 12: `cmd/helix_infra`

Attribute	Value
Transport	CLI
Uses internal/ packages	No — self-contained
Real vs. Simulated	**REAL** — Infrastructure lifecycle manager
Test coverage	`main_test.go`

Key functionality: - Docker Compose lifecycle management (up, down, status, logs) - VM management (spawn, destroy, list, status, SSH) - Health checking - Scaling - Version info

Known gaps: - VM operations require appropriate hypervisor access - No multi-host support - Scale operation is limited

Binary 13: `cmd/e2ee-proxy`

Attribute	Value
Transport	gRPC
Uses internal/ packages	No — uses `pkg/security`
Real vs. Simulated	**REAL** — E2EE proxy

Attribute	Value
Test coverage	`main_test.go`

Key functionality: - End-to-end encryption proxying - Hybrid key exchange (ML-KEM-768 + X25519) - X25519 session key agreement

Known gaps: - No integration with session service - Key rotation not automated - Limited test coverage

Binary 14: `cmd/htmux`

Attribute	Value
Transport	WebSocket
Uses internal/ packages	No — uses `pkg/session`
Real vs. Simulated	**REAL** — Terminal multiplexer
Test coverage	`main.go` tests, `cmd htmux test.go`, `wsstream test.go`, `wsstream integration_test.go`, `hxc1136_e2e_test.go`

Key functionality: - Terminal multiplexer with WebSocket streaming - Client connection management (`client.go`) - WebSocket stream handling (`wsstream.go`) - Terminal I/O (`terminal.go`)

Known gaps: - PTY use-after-close (related to F5) - No terminal resize propagation - Limited to single-node operation

Binary 15: `cmd/helixd`

Attribute	Value
Transport	HTTP
Uses internal/ packages	Minimal
Real vs. Simulated	**STUB** — Hollow shell
Test coverage	`main_test.go`

Key functionality: - Probes etcd, PostgreSQL, Redis connectivity - Serves `/status` endpoint - **Does NOT wire any of the 14 microservices or 178 orphaned packages**

Known gaps: - This is the core daemon but it does almost nothing - No service orchestration - No configuration management - No service dependency resolution

Binary 16: `cmd/helixctl`

Attribute	Value
Transport	gRPC client
Uses internal/ packages	No
Real vs. Simulated	**REAL** — Cobra CLI with gRPC client
Test coverage	`build_test.go`

Key functionality: - `helixctl build submit/status/cancel/logs` — Build service operations - gRPC client connecting to backend services - `list` command intentionally omitted (no backing RPC)

Known gaps: - Only build commands implemented - No node, session, scheduler, or security commands - No config file support

Binary 17: `cmd/helix-setup`

Attribute	Value
Transport	CLI
Uses internal/ packages	Self-contained orchestrator
Real vs. Simulated	**REAL** — Setup orchestrator
Test coverage	`main_test.go`, `orchestrator_test.go`

Key functionality: - First-run setup orchestration - Platform-specific memory detection (`mem_linux.go` , `mem_darwin.go` , `mem_other.go`) - Lifecycle management (`lifecycle.go`) - Setup orchestrator (`orchestrator.go`)

Known gaps: - No interactive setup mode - Limited validation of setup parameters

Binary 18: `cmd/helix-gate`

Attribute	Value
Transport	CLI
Uses internal/ packages	

Attribute	Value
	Uses gate packages (archlint, etcdlint, covgate, qualitygate, phasegate)
Real vs. Simulated	**REAL** — Gate enforcement
Test coverage	`main.go` only

Key functionality: - Runs orphan-prevention gate suite (HXC-940) - Invoked via `make gate-check` - Runs archlint, etcdlint, covgate, qualitygate, phasegate

Known gaps: - Gates not wired into CI (no CI) - Quality gate baseline may be stale - Phase gate criteria not fully defined

Binary 19: `cmd/helix-test`

Attribute	Value
Transport	CLI
Uses internal/ packages	No
Real vs. Simulated	**REAL** — Challenge runner
Test coverage	`main test.go`, `challenge test.go`, `challenge_integration_test.go`

Key functionality: - Challenge execution via HelixQA framework - Challenge definition and registration - Integration test runner

Known gaps: - Challenge bank not populated - No evidence collection pipeline - Untested gate validators (CLAUDE-1 risk)

Binary 20: `cmd/helix-snapshot`

Attribute	Value
Transport	CLI
Uses internal/ packages	No
Real vs. Simulated	**REAL** — Snapshot management
Test coverage	`main_test.go`

Key functionality: - Cluster state snapshot creation - Snapshot restoration

Known gaps: - No incremental snapshots - No snapshot scheduling - Limited storage backend support

Binary 21: `cmd/hxc-registry`

Attribute	Value
Transport	CLI
Uses internal/ packages	Uses `pkg/hxcregistry`
Real vs. Simulated	**REAL** — Registry CLI
Test coverage	`main_test.go`

Key functionality: - HXC ticket management - Artifact registration - Migration tracking

Known gaps: - No bulk import/export - Limited search capabilities

Binary 22: `cmd/dst-sim`

Attribute	Value
Transport	CLI
Uses internal/ packages	No — uses `pkg/testing/dst`
Real vs. Simulated	**REAL** — DST engine
Test coverage	`main_test.go`, `scenario.go`

Key functionality: - Deterministic simulation testing - Scenario definition and execution - Integration with `pkg/testing/dst/engine.go`

Known gaps: - Scenario library is minimal - No integration with real cluster state - Buggify framework limited

Binary 23: `cmd/burst-controller`

Attribute	Value
Transport	HTTP
Uses internal/ packages	No — self-contained
Real vs. Simulated	**REAL** — Burst capacity controller

Attribute	Value
Test coverage	`burst_controller_test.go`

Key functionality: - Burst capacity management - Rate limiting integration - Hysteresis-based burst control

Known gaps: - Not connected to scheduler - No metrics export - Hysteresis parameters not tunable at runtime

Binary 24: `cmd/gpu-pool-manager`

Attribute	Value
Transport	gRPC
Uses internal/ packages	Uses `internal/gpu`
Real vs. Simulated	**REAL** — GPU pool management
Test coverage	`main_test.go`

Key functionality: - GPU resource pool management - MIG profile management - GPU allocation and deallocation

Known gaps: - Requires NVIDIA hardware and drivers - No AMD/Intel backend integration in this binary - Limited fault tolerance

2.2 Internal Package Analysis

`internal/gateway`

Purpose: API gateway with authentication, inference proxy, and OpenAPI specification.

Key structs: - `Gateway` — Main gateway server with HTTP/gRPC handlers - `AuthMiddleware` — JWT authentication middleware - `InferenceProxy` — LLM inference reverse proxy

Key interfaces: - `ServiceRegistry` — Backend service discovery - `AuthProvider` — Authentication abstraction

Real implementation vs. stubs: - ✅ Real HTTP server with routing - ✅ Real JWT middleware (but uses stub tokens from security service — CRIT-1) - ✅ Real inference proxy - ⚠️ OpenAPI spec incomplete - ❌ Not all 14 services routed

Integration points: - `internal/security` — Auth token validation - `internal/session` — Session proxy - `internal/scheduler` — Job proxy

- `pkg/ratelimit` — Rate limiting - `pkg/middleware` — HTTP middleware chain

Known defects: - AUTH-HXC1213: Auth integration test reveals stub token acceptance - Gateway routing incomplete (ARCH-3) - No circuit breaker pattern for backend services

`internal/session`

Purpose: Session management with gRPC server, CRDT state, and stream support.

Key structs: - `Server` — gRPC session server - `StreamHandler` — WebSocket stream attachment

Key interfaces: - `SessionBackend` — Pluggable session backend (native, tmux, screen, zellij)

Real implementation vs. stubs: - ✅ Real gRPC server - ✅ Real stream attachment - ✅ Integration with `pkg/session` CRDT state - ⚠️ Migration strategies (CRIU/DMTCP) return “not implemented” - ❌ PTY double-close hazard (F5)

Integration points: - `pkg/session` — CRDT session state management - `pkg/session/backends` — Terminal backend implementations - `internal/node` — GPU resource queries - `internal/security` — Session authentication

Known defects: - F5: PTY double-close / use-after-close in native backend - Migration stub registration is a PASS-bluff - Stream reconnection unreliable

`internal/scheduler`

Purpose: Omega-model scheduler with gRPC server, preemption, and plugin system.

Key structs: - `Server` — gRPC scheduler server - `Scheduler` — Core scheduling engine - `LifecycleManager` — Job lifecycle tracking

Key interfaces: - `SchedulePlugin` — Scheduling plugin interface - `Preemptor` — Job preemption interface

Real implementation vs. stubs: - ✅ Real gRPC server - ✅ Real omega-model scheduling - ✅ Gang scheduling and preemption - ⚠️ Plugin system not fully populated - ❌ Unbounded placements map (F6)

Integration points: - `pkg/scheduler` — Scheduling algorithms and plugins - `internal/node` — Resource information - `internal/health` — Node health status - `internal/policy` — Policy-based scheduling decisions

Known defects: - F6: Unbounded placements map with no completion path - No backfill scheduling wired - No priority queue wired - No node selector wired

`internal/node`

Purpose: Node management with gRPC server, etcd registry, and resource reporting.

Key structs: - `Server` — gRPC node server - `Registry` — Node registry (etcd-backed) - `EtcdRegistry` — etcd-specific registry implementation

Key interfaces: - `NodeRegistry` — Node registration interface - `ResourceReporter` — Resource reporting interface

Real implementation vs. stubs: - ✅ Real gRPC server - ✅ Real etcd registry - ✅ Real node registration and heartbeat - ✅ GPU device reporting

Integration points: - `pkg/etcd` — etcd key management - `pkg/discovery` — Service discovery - `internal/gpu` — GPU resource detection - `internal/health` — Health self-reporting

Known defects: - HXC-1210: etcd key conflict resolution - HXC-1086/1087: Integration test gaps - Node behavior tests incomplete

`internal/security`

Purpose: Security service with SPIFFE CA, RBAC, identity bindings, and policy enforcement.

Key structs: - `Server` — gRPC security server - `SPIFFECertificateAuthority` — SPIFFE/SPIRE CA implementation - `PolicyEnforcer` — Policy enforcement engine - `IdentityBindings` — Identity-to-role bindings - `Orchestrator` — Security orchestration

Key interfaces: - `CertificateAuthority` — CA abstraction - `Authorizer` — Authorization interface

Real implementation vs. stubs: - ✅ Real SPIFFE/SPIRE CA (`spiffe_ca.go`) - ✅ Real RBAC with scopes (`scopes.go`) - ✅ Real identity bindings (`identity_bindings.go`) - ✅ Real policy enforcement (`policy_enforcer.go`) - ❌ **STUB JWT tokens** — no signature verification (CRIT-1) - ⚠️ Vault integration exists but may not be fully wired

Integration points: - `pkg/security` — TLS, Vault, SPIFFE client - `pkg/jwt` — JWT token handling - `pkg/hybridkex` — Post-quantum key exchange - `internal/policy` — Policy evaluation - `pkg/spiffefed` — SPIFFE federation

Known defects: - CRIT-1: Stub JWT tokens with no signature verification - RBAC scopes not enforced at gateway level - mTLS e2e incomplete

(HXC-600) - SPIFFE-CA unit tests pass but integration with real SPIRE server untested

`internal/health`

Purpose: Health checking service with aggregation, rollup, and platform-specific system calls.

Key structs: - `Server` — gRPC/HTTP health server - `Aggregator` — Health status aggregation - `Rollup` — Health rollup computation - `Checker` — Individual health checker

Key interfaces: - `HealthTarget` — Health check target interface - `HealthAggregator` — Aggregation interface

Real implementation vs. stubs: - ✅ Real health checking - ✅ Real aggregation and rollup - ✅ Platform-specific system calls (Unix/Windows) - ❌ **HTTP-only, no gRPC integration to gateway** (CRIT-2) - ⚠️ Watch mechanism incomplete

Integration points: - `pkg/health` — Health gRPC service, miner, rollup - `internal/gateway` — (should be, but isn't) gateway health proxy - All services — Health check targets

Known defects: - CRIT-2: HTTP-only, no gRPC integration - Watch mechanism not connected to event bus - No health history persistence - Aggregator lacks real service discovery

`internal/build`

Purpose: Build service with orchestration, Podman execution, and worker pool.

Key structs: - `Server` — gRPC build server - `Orchestrator` — Build orchestration engine - `PodmanExecutor` — Podman container build executor - `ExecBuilder` — Go build executor - `Worker` — Build worker - `SimulatedBuilder` — Simulated builder (for testing)

Key interfaces: - `Builder` — Build execution interface - `BuildExecutor` — Executor abstraction

Real implementation vs. stubs: - ✅ Real Podman executor - ✅ Real Go build executor - ✅ Real orchestrator - ⚠️ `SimulatedBuilder` registered alongside real builders - ❌ **Wrong imports** bypass orchestration layer (CRIT-3)

Integration points: - `pkg/build` — Build platform, manifest, cache - `internal/scheduler` — Job queue integration - `pkg/events` — Build event streaming

Known defects: - CRIT-3: Wrong import paths bypass orchestration - SimulatedBuilder registered in production path - Podman requires running

daemon - No artifact storage integration - Stream output not connected to gateway WebSocket

`internal/policy`

Purpose: Policy service with OPA engine, decision logging, and scheduling policy.

Key structs: - `Engine` — OPA policy evaluation engine - `DecisionLog` — Decision logging to PostgreSQL - `SchedulingPolicy` — Scheduling-specific policy

Key interfaces: - `PolicyEngine` — Policy evaluation interface - `DecisionLogger` — Decision logging interface

Real implementation vs. stubs: - ✅ Real OPA engine (`engine.go`) - ✅ Real decision logging (`decision_log.go`) - ✅ Real scheduling policy (`scheduling_policy.go`)

Integration points: - `internal/security` — Policy enforcement - `internal/scheduler` — Scheduling decisions - PostgreSQL — Decision log storage - OPA — Policy evaluation

Known defects: - Decision log schema may not match migration chain - OPA policy bundles not automatically loaded - No hot-reload of policies

`internal/llm`

Purpose: LLM service with model management, advisory generation, and verification.

Key structs: - `Manager` — LLM model manager - `Advisory` — Advisory generation - `Verifier` — Response verification

Key interfaces: - `LLMProvider` — LLM backend interface - `AdvisoryGenerator` — Advisory generation interface

Real implementation vs. stubs: - ✅ Real manager logic - ✅ Real advisory generation - ⚠️ Verifier is heuristic-based - ❌ No actual LLM backend connection

Integration points: - `internal/advisory` — Advisory routing - `internal/gateway` — Inference proxy - `pkg/quantization` — Model quantization

Known defects: - No actual LLM backend (OpenAI/Anthropic/etc.) - Verification is not cryptographically sound - Advisory routing not connected to gateway

`internal/advisory`

Purpose: Advisory service with gRPC server and manager.

Key structs: - `Server` — gRPC advisory server - `Manager` — Advisory management

Real implementation vs. stubs: - ✅ Real gRPC server - ✅ Real advisory management - ⚠️ Simple rule-based logic

Known defects: - Not ML-driven - No external advisory source integration - gRPC tests are unit-level only

`internal/messaging`

Purpose: Event bus with etcd topics, message queue, and delivery guarantees.

Key structs: - `Bus` — Event bus implementation - `Queue` — Message queue - `Message` — Message type

Real implementation vs. stubs: - ✅ Real in-memory event bus - ✅ Real message queue with delivery tracking - ⚠️ etcd topics integration incomplete

Known defects: - Bus introspection limited - No guaranteed ordering - Delivery tracking may have race conditions

`internal/backup`

Purpose: Backup service with PostgreSQL integration.

Key structs: - `BackupManager` — Backup orchestration

Real implementation vs. stubs: - ✅ Real backup logic - ✅ Integration tests with PostgreSQL

Known defects: - No incremental backup - No backup scheduling - Limited storage backend support

`internal/chaos`

Purpose: Chaos engineering framework for fault injection.

Key structs: - `ChaosEngine` — Chaos fault injection engine

Real implementation vs. stubs: - ✅ Real chaos engine - ✅ Fake cluster for testing

Known defects: - Limited fault types - No integration with DST engine - Fake cluster may not represent real topology

`internal/console`

Purpose: Console hardware detection and management (PS4/PS5/RPi/RK3588).

Key structs: - `Detector` — Hardware detection - `BootCoordinator` — Boot sequence coordination - `GPUDetector` — GPU detection for consoles - `ThermalMonitor` — Thermal monitoring - `TrustManager` — Trust establishment - `Register` — Hardware register access

Real implementation vs. stubs: - ✅ Real hardware detection from `/proc/cpuinfo` and device tree - ✅ Test fixtures for PS4 Jaguar, PS5 Zen2, RPi, RK3588, x86, generic - ⚠️ Some detection is Linux-specific

Known defects: - macOS detection limited (no device-tree equivalent) - Boot coordinator not integrated with node service - Thermal monitoring not connected to health service

`internal/federation`

Purpose: Federation across geographic cells with suspicion-based trust.

Key structs: - `Hub` — Federation hub - `Selector` — Cell selection - `Policy` — Federation policy

Real implementation vs. stubs: - ✅ Real federation logic - ✅ Suspicion-based trust model - ✅ Chaos tests for federation

Known defects: - Not wired into any binary - No real network federation tested - Aggregate function may have edge cases

`internal/gpu`

Purpose: GPU management with detection, reservation, MIG, backend registry, and attestation.

Key structs: - `Manager` — GPU resource manager - `Backend` — GPU backend interface - `Reservation` — GPU reservation tracking - `MIGManager` — NVIDIA MIG profile management - `BackendRegistry` — Backend registration and selection - `AttestHook` — GPU attestation hook - `Monitor` — GPU health monitoring - `NvidiaParser` — NVIDIA SMI output parser

Real implementation vs. stubs: - ✅ Real NVIDIA GPU detection and parsing - ✅ Real MIG management - ✅ Real Apple GPU backend (`backend_darwin_apple.go`) - ✅ Real GPU sharing (`backend_sharing.go`) - ✅ Real HelixPoW attestation - ⚠️ AMD/Intel backends may be stubs - ✅ Platform-specific detection (Linux, Darwin, Other)

Known defects: - AMD and Intel backends need verification - Backend sharing test coverage limited - GPU attestation requires real hardware - macOS GPU integration test needs Apple hardware

`internal/wireguard`

Purpose: WireGuard mesh networking with monitoring and hardening.

Key structs: - `Mesh` — WireGuard mesh management - `Peer` — Peer configuration - `Monitor` — WireGuard link monitor

Real implementation vs. stubs: - ✅ Real WireGuard management - ✅ Real peer configuration - ✅ Real monitoring - ✅ Hardening tests

Known defects: - Monitor hardening tests are limited - No integration with `pkg/wireguard` mesh coordinator - Linux-specific features need macOS equivalents

`internal/costbroker`

Purpose: Cost brokering with source management.

Key structs: - `Broker` — Cost broker - `Source` — Cost data source

Real implementation vs. stubs: - ✅ Real broker logic

Known defects: - Not wired into any binary - Limited cost source integration

`internal/schema`

Purpose: Database schema management with drift guard.

Key structs: - Schema management types

Real implementation vs. stubs: - ✅ Real schema management - ✅ Real drift guard test

Known defects: - HXC-1639: Primary schema diverges from migration chain - Drift guard not wired into CI

`internal/verifier`

Purpose: Response verification.

Key structs: - `Verifier` — Response verifier

Real implementation vs. stubs: - ✅ Real verification logic

Known defects: - Limited verification methods - Not wired into build pipeline

`internal/trust`

Purpose: Trust scoring for nodes and services.

Key structs: - `Scorer` — Trust score calculator

Real implementation vs. stubs: - ✅ Real scoring logic

Known defects: - Not wired into security service - Score weights not configurable

---

Chapter 3: Foundation Package (pkg/) Analysis

3.1 Overview

The `pkg/` directory contains approximately 212 packages. Of these, only ~30 are reachable from any binary. The remaining ~178 are orphaned — implemented but not wired into any running application. This section provides a comprehensive analysis of each package.

3.2 Anti-Bluff Audit Results (30 packages audited)

The initial anti-bluff audit covered 30 core packages:

#	Package	Verdict	Risk Level	Key Finding
1	`pkg/backoff`	❌ FAIL	MEDIUM	No mutation tests
2	`pkg/classads`	❌ FAIL	MEDIUM	No mutation tests, minimal coverage
3	`pkg/config`	❌ FAIL	HIGH	STUB BLUFF: Load() returns hardcoded defaults, no env reading
4	`pkg/context`	❌ FAIL	MEDIUM	No mutation tests
5	`pkg/crypto`	❌ FAIL	HIGH	No mutation tests, no fuzz, TestHash only checks length
6	`pkg/discovery`	✅ PASS	LOW	21 tests, 9 mutation tests, real TTL checker

#	Package	Verdict	Risk Level	Key Finding
7	`pkg/errors`	✅ PASS	LOW	18 tests, 8 mutation tests, real stack traces
8	`pkg/events`	❌ FAIL	MEDIUM	No mutation tests, flaky time.Sleep in tests
9	`pkg/grpcutil`	❌ FAIL	HIGH	STUB BLUFF: No-op pass-through interceptors
10	`pkg/health`	❌ FAIL	MEDIUM	No mutation tests, minimal coverage
11	`pkg/infra`	❌ FAIL	HIGH	STUB BLUFF: Pure in-memory simulation, no real orchestration
12	`pkg/jwt`	❌ FAIL	HIGH	STUB BLUFF: Only splits strings, no signature verification
13	`pkg/leader`	❌ FAIL	HIGH	STUB BLUFF: Single-process atomic flag, not distributed
14	`pkg/log`	✅ PASS	LOW	21 tests, 8 mutation tests, real slog backend
15	`pkg/lru`	❌ FAIL	MEDIUM	No mutation tests
16	`pkg/metrics`	❌ FAIL	MEDIUM	No mutation tests
17	`pkg/middleware`	❌ FAIL	HIGH	STUB BLUFF: LoggingMiddleware is a no-op
18	`pkg/netutil`	❌ FAIL	MEDIUM	No mutation tests
19	`pkg/pubsub`	❌ FAIL	MEDIUM	No mutation tests
20	`pkg/ratelimit`	❌ FAIL	MEDIUM	

#	Package	Verdict	Risk Level	Key Finding
				No mutation tests, flaky time.Sleep tests
21	`pkg/retry`	❌ FAIL	MEDIUM	No mutation tests
22	`pkg/semaphore`	❌ FAIL	MEDIUM	No mutation tests
23	`pkg/serde`	❌ FAIL	MEDIUM	No mutation tests
24	`pkg/session`	✅ PASS	LOW	67 tests, many mutation tests, real CRDT with LWW
25	`pkg/swim`	✅ PASS	LOW	54 tests, 22 mutation tests, real UDP transport
26	`pkg/tracing`	❌ FAIL	HIGH	STUB BLUFF: Hardcoded trace IDs, no propagation
27	`pkg/validator`	❌ FAIL	MEDIUM	No mutation tests
28	`pkg/websocket`	❌ FAIL	HIGH	STUB BLUFF: Upgrade returns nil, no handshake
29	`pkg/wireguard`	⚠️ CONDITIONAL	MEDIUM	Real on Linux+root, 7/13 tests skipped on macOS
30	`pkg/workerpool`	❌ FAIL	MEDIUM	No mutation tests

Stub Bluffing Hall of Shame

These 8 packages claim to implement features but do not actually work for end users:

Package	Claimed Feature	Actual Reality	Impact
`pkg/config`			Any code relying on

Package	Claimed Feature	Actual Reality	Impact
	Load config from environment	Returns hardcoded struct	config from env gets wrong values
`pkg/grpcutil`	gRPC interceptors	No-op pass-through stubs	No observability, no auth, no logging in gRPC calls
`pkg/infra`	Infrastructure orchestration	In-memory simulation only	No real Docker/ VM/cloud management
`pkg/jwt`	JWT parsing	Splits strings, no verification	Security vulnerability: accepts any token
`pkg/leader`	Leader election	Single-process atomic flag	No distributed consensus
`pkg/ middleware`	HTTP logging middleware	No-op pass-through	No request logging
`pkg/tracing`	Distributed tracing	Hardcoded trace IDs	No actual tracing, no propagation
`pkg/websocket`	WebSocket upgrade	Returns nil, no handshake	No real WebSocket support

3.3 Detailed Package Analysis (Extended)

`pkg/scheduler` — Omega-Model Scheduler

Purpose: Comprehensive scheduling engine with plugins, gang scheduling, backfill, and preemption.

Implementation completeness: Real implementation, partially wired.

Key files: - `scheduler.go` — Core scheduling engine with omega-model optimistic concurrency - `plugins.go` — Plugin registration and execution framework - `queue.go` — Priority queue for scheduling - `gang preempt.go` — Gang scheduling with preemption support - `classad match.go` — ClassAd-based resource matching - `classad_named plugins.go` — Named plugin system for ClassAd expressions - `tier_filter.go` — Tier-based node filtering -

`tier_power.go` — Tier power management - `edgeaware.go` — Edge-aware scheduling - `thermal.go` — Thermal-aware scheduling - `cost_gpu.go` — Cost-aware GPU scheduling - `attestation.go` — Attestation-based scheduling - `handheld.go` — Handheld device scheduling - `latency_spot.go` — Latency spot scheduling

Test coverage: Extensive — includes HXC-specific integration tests (hxc1080, hxc1081, hxc1082, hxc1083), stress/chaos tests, and behavioral tests.

Anti-bluff assessment: PASS — real scheduling logic with extensive testing.

Known issues: - F6: Unbounded placements map - Scheduler helpers (backfill, priorityqueue, nodeselector) not wired - No real resource data integration

`pkg/swim` — SWIM Gossip Protocol

Purpose: SWIM (Scalable Weakly-consistent Infection-style Process Group Membership) protocol implementation.

Implementation completeness: Real implementation with UDP transport, failure detection, and gossip dissemination.

Key files: - `protocol.go` — Core SWIM protocol with join/leave/probe operations - `failure detector.go` — Failure detection with suspicion mechanism - `phi_accrual.go` — Phi accrual failure detector - `gossip.go` — Gossip message dissemination - `member.go` — Member state management - `transport.go` — Transport abstraction - `simtransport.go` — Simulated transport for testing - `hierarchical.go` — Hierarchical SWIM for large clusters - `suspicion.go` — Suspicion mechanism - `clock.go` — Logical clock for ordering - `prober.go` — Prober for member health checking

Test coverage: 54+ tests including 22 mutation tests, integration tests, concurrency stress/chaos tests, and lifecycle tests.

Anti-bluff assessment: PASS — strongest package in the audit.

Known issues: - F1: Double-close panic in Stop/Leave - F2: Data race on Member.State - F3: Untracked goroutine in probeRandomMember - F4: Lock-order fragility in Confirm

`pkg/session` — CRDT Session Management

Purpose: CRDT-based session management with LWW conflict resolution, migration, and checkpoint support.

Implementation completeness: Real implementation with multiple backends.

Key files: - `crdt.go` — CRDT state management with LWW semantics - `convergence.go` — State convergence logic - `lifecycle.go` — Session lifecycle management - `manager.go` — Session manager - `migration.go` — Migration strategy planner - `checkpoint.go` — Checkpoint support - `backend_adapter.go` — Backend adapter pattern - `types.go` — Core types - `backends/native.go` — Native PTY backend - `backends/tmux.go` — Tmux backend - `backends/tmux_cc.go` — Tmux control mode backend - `backends/screen.go` — GNU Screen backend - `backends/zellij.go` — Zellij backend

Test coverage: 67+ tests with mutation coverage, fuzz tests, and stress/chaos tests.

Anti-bluff assessment: PASS — largest and most thoroughly tested package.

Known issues: - F5: PTY double-close / use-after-close in native backend - Migration strategies (CRIU/DMTCP) return "not implemented" — PASS-bluff - GetResourceUsage is a documented placeholder returning zeros

`pkg/wireguard` — WireGuard Mesh

Purpose: WireGuard mesh networking with configuration, key rotation, NAT traversal, and policy management.

Implementation completeness: Real implementation with platform-specific support.

Key files: - `manager.go` — WireGuard manager with wgctrl client - `mesh.go` — Mesh coordination - `meshconfig.go` — Mesh configuration generation - `config.go` — WireGuard interface configuration - `configgen.go` — Configuration generation - `keyrotation.go` — Key rotation with configurable intervals - `holepunch.go` — UDP hole punching for NAT traversal - `policy.go` — WireGuard policy enforcement - `stun.go` — STUN server for external address discovery - `nat_traversal.go` — NAT traversal strategies - `netstack_darwin.go` — macOS userspace WireGuard (wireguard-go) - `teardown.go` — Clean teardown logic

Test coverage: 13+ tests, but 7 are skipped on macOS/non-root. Integration tests exist for mesh config, STUN hole punching, and policy.

Anti-bluff assessment: CONDITIONAL PASS — real on Linux+root, but test gap on non-Linux platforms.

Known issues: - 7/13 tests skipped on macOS and non-root (CLAUDE-2 concern) - NAT traversal stubs: DiscoverExternalAddress returns "not available", UPnP returns "not implemented" - No mutation tests

`pkg/events` — Event Bus

Purpose: Event bus with multiple backends (NATS, Helix built-in), Avro wire format, and stream configuration.

Implementation completeness: Real implementation with Avro serialization.

Key files: - `helix_backend.go` — Built-in event backend - `nats_backend.go` — NATS JetStream backend - `avro.go` — Avro schema definition - `avro_wire.go` — Avro wire format serialization - `avro_event_wire.go` — Event-specific Avro wire format - `avro_event_schemas.go` — Event Avro schemas - `stream_config.go` — Stream configuration management

Test coverage: Unit and integration tests for all backends and wire formats.

Anti-bluff assessment: FAIL (in initial audit) but improved — real backends exist now.

Known issues: - NATS integration requires running NATS server - Avro schema evolution not tested - Stream config may have race conditions

`pkg/security` — Security Toolkit

Purpose: TLS management, Vault integration, SPIFFE client, and secret injection.

Key files: - `tls.go` — TLS certificate management - `vault.go` — HashiCorp Vault integration - `vault_api_client.go` — Vault API client - `spiffe.go` — SPIFFE/SPIRE client - `secret_injector.go` — Secret injection from Vault

Test coverage: Unit tests + integration tests for Vault, SPIFFE, and TLS.

Known issues: - Vault integration requires running Vault server - SPIFFE integration requires running SPIRE server - Secret injector not wired into all services

`pkg/hybridkex` — Post-Quantum Key Exchange

Purpose: Hybrid key exchange combining ML-KEM-768 (post-quantum) with X25519 (classical).

Key files: - `hybridkex.go` — Hybrid key exchange implementation

Test coverage: Unit tests + fuzz test (`fuzz_test.go`).

Anti-bluff assessment: Real implementation with fuzz testing — above average.

Known issues: - No integration test with actual network transport - ML-KEM-768 implementation needs NIST compliance verification

`pkg/resources` — System Resource Monitoring

Purpose: Cross-platform resource monitoring (CPU, memory, GPU, DRM, cgroups) with platform-specific implementations.

Key files: - `proc_linux.go` — Linux /proc filesystem reader - `proc_darwin.go` — macOS sysctl/vm_stat reader - `cgroup_v2.go` — cgroup v2 resource tracking - `drm_linux.go` — Linux DRM/GPU sysfs reader - `drm_darwin.go` — macOS GPU via system_profiler - `accel_linux.go` — Linux hardware accelerator monitoring - `accel_darwin.go` — macOS accelerator monitoring - `types.go` — Shared resource types - `aggregator.go` — Resource aggregation

Test coverage: Unit tests per platform + integration tests + test fixtures for cgroup and Darwin data.

Anti-bluff assessment: PASS — real platform-specific implementations with test fixtures.

Known issues: - `proc_mock.go` exists (mock for non-Linux) — CLAUDE-2 concern - `drm_other.go` — stub for unsupported platforms - Test fixtures need updating for new hardware

`pkg/etcd` — etcd Key Management

Purpose: etcd key pattern definitions and integration helpers.

Key files: - `keys.go` — Key pattern definitions - `package.go` — Package documentation

Test coverage: Unit + integration tests for key patterns and etcd operations.

Known issues: - HXC-1115: Key pattern refactoring - HXC-1076: Integration test gaps

`pkg/crdt` — CRDT Primitives

Purpose: CRDT (Conflict-free Replicated Data Type) implementations: G-Counter, PN-Counter, OR-Set, LWW-Map, vector clocks, Merkle trees.

Key files: - `crdt.go` — Core CRDT types - `vectorclock.go` — Vector clock implementation - `merkle/merkle.go` — Merkle tree for anti-entropy - `bench_test.go` — Benchmarks

Test coverage: Unit tests for all CRDT types + benchmarks.

Known issues: - Not wired into any binary - No integration test with distributed state

`pkg/mvcc` — Multi-Version Concurrency Control

Purpose: MVCC implementation with B-tree storage.

Key files: - `mvcc.go` — MVCC transaction management - `btree.go` — B-tree storage engine

Test coverage: Unit tests.

Known issues: - Not wired into any binary - No concurrent transaction testing

`pkg/stonith` — Shoot-The-Other-Node-In-The-Head

Purpose: STONITH implementation with multiple fencing backends.

Key files: - `stonith.go` — STONITH interface and manager - `ipmi.go` — IPMI fencing - `sbd.go` — SBD (Shared Block Device) fencing - `cloud.go` — Cloud provider fencing - `noop.go` — No-op fencing (for testing)

Test coverage: Unit tests per backend + cloud-specific tests.

Known issues: - Not wired into any binary - IPMI requires real BMC hardware - Cloud fencing requires cloud credentials - SBD requires shared block device

`pkg/cloudspot` — Cloud Instance Metadata

Purpose: Cloud instance metadata detection for AWS, GCP, and Azure.

Key files: - `imds_aws.go` — AWS IMDS integration - `imds_gcp.go` — GCP metadata integration - `imds_azure.go` — Azure IMDS integration - `fake.go` — Fake metadata for testing

Test coverage: Unit tests with IMDS mocking.

Known issues: - Not wired into any binary - Requires actual cloud instances for integration testing

`pkg/storage` — Storage Backends

Purpose: Storage abstraction with Redis and S3 backends.

Key files: - `storage.go` — Storage interface - `redis_store.go` — Redis storage backend - `redis_client.go` — Redis client wrapper - `s3.go` — S3 storage backend

Test coverage: Unit + integration tests for both backends.

Known issues: - Not wired into any binary - S3 integration requires AWS credentials - Redis integration requires running Redis

`pkg/federation` — Federation

Purpose: Federation across geographic cells with suspicion-based trust.

Key files: - `registry.go` — Cell registry - `cell.go` — Cell management - `suspicion.go` — Suspicion-based trust - `aggregate.go` — Result aggregation - `chaos_test.go` — Chaos testing for federation

Test coverage: Unit + chaos tests.

Known issues: - Not wired into any binary - No real network federation tested

`pkg/marketplaceadapter` — Marketplace Adapters

Purpose: Marketplace adapter for Akash network and other decentralized compute providers.

Key files: - `marketplaceadapter.go` — Adapter interface - `akash.go` — Akash network adapter

Test coverage: Unit tests for Akash adapter.

Known issues: - Not wired into any binary - Requires Akash network access for integration

`pkg/quantization` — Model Quantization

Purpose: LLM model quantization with AWQ support and advisory.

Key files: - `quantization.go` — Quantization framework - `awq.go` — AWQ quantization - `advisor.go` — Quantization advisor

Test coverage: Unit tests for AWQ and advisor.

Known issues: - Not wired into LLM service - No real model quantization tested

`pkg/chutes` — Chutes Client

Purpose: Client for Chutes AI inference platform.

Key files: - `client.go` — Chutes API client - `stream.go` — Streaming inference - `config.go` — Configuration - `envelope.go` — Request/response envelope - `attest.go` — Attestation

Test coverage: Unit tests for client, stream, config, envelope, attest.

Known issues: - Not wired into any binary - Requires Chutes API access

`pkg/covgate` — Coverage Gate

Purpose: Coverage gate enforcement with parsing and audit.

Key files: - `covgate.go` — Coverage gate logic - `parse.go` — Coverage profile parsing - `audit.go` — Audit trail

Test coverage: Unit tests + fuzz test for parser.

Known issues: - Not wired into `make` targets (PRR item 7) - 80% threshold not enforced - Parse fuzz testing incomplete

`pkg/phasegate` — Phase Gate

Purpose: Phase gate enforcement for release readiness.

Key files: - `phasegate.go` — Phase gate logic

Test coverage: Unit tests.

Known issues: - Criteria not fully defined - Not integrated with `helix-gate` binary

`pkg/archlint` — Architecture Linter

Purpose: Architecture linting to prevent layer violations.

Key files: - `archlint.go` — Linter implementation

Test coverage: Unit tests.

Known issues: - Not wired into CI - Layer rules may be incomplete

`pkg/etcdlint` — etcd Key Linter

Purpose: Linting for etcd key patterns and usage.

Key files: - `etcdlint.go` — Linter implementation

Test coverage: Unit tests.

Known issues: - Not wired into CI

`pkg/qualitygate` — Quality Gate

Purpose: Quality gate enforcement with baseline metrics.

Key files: - Baseline metrics in `test/qualitygate/baseline_metrics.json`

Known issues: - Baseline metrics may be stale - Not enforced in build pipeline

`pkg/testing/` — Test Infrastructure

Purpose: Testing infrastructure packages for DST, chaos, evidence collection, and scenario management.

Sub-packages: - `testing/dst` — Deterministic Simulation Testing engine - `testing/dstscale` — DST scale testing - `testing/dstcompress` — DST compression - `testing/dstworkload` — DST workload generation - `testing/chaos` — Chaos fault injection framework - `testing/evidence` — Evidence collection and storage - `testing/scenario` — Scenario definition and execution - `testing/runner` — Test runner framework - `testing/snapshot` — Snapshot-based testing - `testing/regression` — Regression testing with Welch's t-test - `testing/device` — Device provisioning and hardware testing - `testing/turmoil` — Turmoil-based network testing - `testing/instance` — Instance management for testing - `testing/sessionfsm` — Session FSM testing

Known issues: - DST engine scenario library minimal - Evidence collection not connected to HelixQA - Device provisioning requires real hardware - Regression framework not integrated with CI

3.4 Orphaned Package Census

The following packages are implemented but have **zero importers from any binary**:

1. `pkg/anticheat` — Anti-cheat token verification
2. `pkg/attestadmit` — Attestation-based admission
3. `pkg/auditproof` — Audit proof generation
4. `pkg/backfill` — Backfill scheduling
5. `pkg/billingfsm` — Billing finite state machine
6. `pkg/burst` — Burst capacity management
7. `pkg/bursthysteresis` — Burst hysteresis control
8. `pkg/capability` — Capability-based access control
9. `pkg/cellmesh` — Cell mesh networking
10. `pkg/chaosexp` — Chaos experiment framework
11. `pkg/classads` — ClassAd expression parser and evaluator
12. `pkg/cloudspot` — Cloud instance metadata
13. `pkg/costrouter` — Cost-based routing
14. `pkg/costsched` — Cost-aware scheduling
15. `pkg/costtracker` — Cost tracking
16. `pkg/covgate` — Coverage gate enforcement
17. `pkg/crdt` — CRDT primitives
18. `pkg/databrowser` — Data browsing
19. `pkg/deltacrdt` — Delta-state CRDTs
20. `pkg/devicecatalog` — Device catalog
21. `pkg/deviceplugin` — Device plugin framework
22. `pkg/deviceprofile` — Device profiling
23. `pkg/doublecrypt` — Double encryption
24. `pkg/epochresolve` — Epoch resolution

25. `pkg/etcd` — etcd key management
26. `pkg/etcdlint` — etcd key linter
27. `pkg/ewmarank` — EWM ranking
28. `pkg/exportcontrol` — Export control
29. `pkg/failconfirm` — Failure confirmation
30. `pkg/fallbackchain` — Fallback chain routing
31. `pkg/fedtrust` — Federation trust
32. `pkg/fiber` — Fiber admission control
33. `pkg/fmea` — Failure Mode and Effects Analysis
34. `pkg/forecast` — Resource forecasting
35. `pkg/gpuattest` — GPU attestation
36. `pkg/gpucatalog` — GPU catalog
37. `pkg/gpupool` — GPU pooling
38. `pkg/gputopo` — GPU topology
39. `pkg/grafanadash` — Grafana dashboard generation
40. `pkg/gravaladmit` — GraVal admission
41. `pkg/gravalverify` — GraVal verification
42. `pkg/hashslot` — Hash slot routing
43. `pkg/headersanitize` — Header sanitization
44. `pkg/heartbeatcoalescer` — Heartbeat coalescing
45. `pkg/helixnet` — Network simulation
46. `pkg/helixtask` — Task management
47. `pkg/hlc` — Hybrid logical clocks
48. `pkg/hybridtco` — Hybrid TCO calculation
49. `pkg/idempotent` — Idempotent operation tracking
50. `pkg/imagepolicy` — Image policy enforcement
51. `pkg/inferenceproxy` — Inference proxy
52. `pkg/jobadmit` — Job admission control
53. `pkg/kraft` — Unikraft integration
54. `pkg/latencysched` — Latency-aware scheduling
55. `pkg/llmfailover` — LLM failover
56. `pkg/local` — Local execution
57. `pkg/marketplaceadapter` — Marketplace adapters
58. `pkg/metering` — Metering
59. `pkg/metrics` — Metrics collection (partially wired)
60. `pkg/middleware` — HTTP middleware
61. `pkg/modelintegrity` — Model integrity verification
62. `pkg/modelretry` — Model retry logic
63. `pkg/mvcc` — Multi-version concurrency control
64. `pkg/nattraversal` — NAT traversal
65. `pkg/nodeselector` — Node selection
66. `pkg/offlinesync` — Offline synchronization
67. `pkg/openapivalidate` — OpenAPI validation
68. `pkg/passthrough` — Passthrough proxy
69. `pkg/phase7matrix` — Phase 7 completion matrix
70. `pkg/pool` — Resource pooling
71. `pkg/porcupine` — Porcupine testing framework
72. `pkg/priorityqueue` — Priority queue
73. `pkg/providerchain` — Provider chain routing
74. `pkg/pubsub` — Pub/sub messaging
75. `pkg/quantization` — Model quantization
76. `pkg/rebalance` — Workload rebalancing

77. `pkg/revenueopt` — Revenue optimization
78. `pkg/ringavg` — Ring buffer average
79. `pkg/sandbox` — Sandboxed execution
80. `pkg/scan` — Security scanning
81. `pkg/smartrouter` — Smart routing
82. `pkg/slotmigration` — Slot migration
83. `pkg/spiffefed` — SPIFFE federation
84. `pkg/splitbrain` — Split-brain detection
85. `pkg/splitbrainalert` — Split-brain alerting
86. `pkg/stats` — Statistical analysis (Welch's t-test)
87. `pkg/storage` — Storage backends
88. `pkg/stressmark` — Stress marking
89. `pkg/thermalwarm` — Thermal warming
90. `pkg/tierdef` — Tier definition
91. `pkg/tieredcache` — Tiered caching
92. `pkg/timefault` — Time fault injection
93. `pkg/tofu` — TOFU (Trust On First Use)
94. `pkg/watchmanager` — Watch management
95. `pkg/watchtower` — Watchtower monitoring
96. `pkg/workclaim` — Work claiming
97. `pkg/workloadrouter` — Workload routing
98. `pkg/x25519session` — X25519 session key agreement
99. `pkg/anonymize` — Data anonymization
100. `pkg/gitops` — GitOps integration
101. `pkg/compliancedoc` — Compliance documentation
102. `pkg/edge` — Edge computing support
103. `pkg/edgefusion` — Edge data fusion
104. `pkg/edgeheartbeat` — Edge heartbeat
105. `pkg/edgeverify` — Edge verification
106. `pkg/federation` — Federation management
107. `pkg/flowcontrol` — Flow control
108. `pkg/agentprovision` — Agent provisioning
109. `pkg/computeproto` — Compute protocol (FlatBuffers)
110. `pkg/wasm` — WebAssembly runtime
111. `pkg/hxcregistry` — HXC registry (SQLite)
112. `pkg/gpu` — GPU management
113. `pkg/dst` — DST framework
114. `pkg/burstcapacity` — Burst capacity

…and approximately 64 additional packages including testing infrastructure, niche utilities, and specialized algorithms.

Chapter 4: Concurrency Hazard Analysis

4.1 Confirmed Hazards (F1–F10)

F1: Double-Close Panic in SWIM Protocol

Attribute	Value
Location	`pkg/swim/protocol.go` — Stop() and Leave() methods
Class	Double-close panic
Severity	Major
Confidence	High
Status	IN PROGRESS
Race condition type	Channel double-close

Detailed analysis: The `Stop()` method closes the internal `done` channel. The `Leave()` method also closes the same channel. If both are called, or if `Stop()` is called twice, the second close panics. Additionally, `probeRandomMember()` may be running when `Stop()` is called, leading to a send-on-closed-channel panic.

```
// pkg/swim/protocol.go (simplified)
func (p *Protocol) Stop() {
    close(p.done) // PANIC if already closed
    p.cancel()
}

func (p *Protocol) Leave() {
    close(p.done) // PANIC if Stop() already called
    // ... graceful leave logic never reached
}
```

Remediation: 1. Use `sync.Once` for channel close 2. Add atomic flag to prevent double-close 3. Use `select` with default case for channel sends 4. Ensure all goroutines are waited on before close

```
type Protocol struct {
    closeOnce sync.Once
```

```
    done      chan struct{}
    // ...
}

func (p *Protocol) Stop() {
    p.closeOnce.Do(func() {
        close(p.done)
    })
    p.cancel()
    p.wg.Wait() // wait for all goroutines
}
```

F2: Data Race on Member.State

Attribute	Value
Location	`pkg/swim/protocol.go` — HealthyMembers() and Members()
Class	Data race on shared state
Severity	Major
Confidence	High
Status	IN PROGRESS
Race condition type	Read-write race

Detailed analysis: `Member.State` is accessed concurrently by the protocol goroutine (which updates it) and the `HealthyMembers()` / `Members()` methods (which read it). There is no mutex protecting `State`.

```
// pkg/swim/member.go (simplified)
type Member struct {
    ID    string
    State MemberState  // accessed without synchronization!
    // ...
}

// Written by protocol goroutine:
func (p *Protocol) handleSuspect(m Member) {
    m.State = StateSuspect // WRITE
}

// Read by API callers:
func (p *Protocol) HealthyMembers() []Member {
```

```
    var healthy []Member
    for _, m := range p.members {
        if m.State == StateAlive { // READ - RACE!
            healthy = append(healthy, m)
        }
    }
    return healthy
}
```

Remediation: 1. Protect `Member.State` with a mutex 2. Or use atomic operations for state 3. Copy members before returning to callers

F3: Untracked Goroutine in probeRandomMember

Attribute	Value
Location	`pkg/swim/protocol.go` — probeRandomMember()
Class	Untracked goroutine
Severity	Minor
Confidence	High
Status	IN PROGRESS

Detailed analysis: `probeRandomMember()` launches goroutines for indirect probing without tracking them in a `WaitGroup`. This can lead to goroutine leaks if the protocol is stopped while probes are in flight.

Remediation: 1. Add probe goroutines to the protocol's `WaitGroup` 2. Ensure context cancellation propagates to probe goroutines

F4: Lock-Order Fragility in Failure Detector

Attribute	Value
Location	`pkg/swim/failure_detector.go` — Confirm()
Class	Lock-order fragility
Severity	Major
Confidence	Medium
Status	IN PROGRESS

Detailed analysis: The `Confirm()` method acquires locks in a different order than other methods, potentially causing deadlocks under high contention.

Remediation: 1. Establish and document lock ordering 2. Use `go test -race` to verify 3. Consider lock-free data structures for hot paths

F5: PTY Double-Close / Use-After-Close

Attribute	Value
Location	`pkg/session/backends/native.go`
Class	Resource double-close / use-after-close
Severity	Major
Confidence	High
Status	IN PROGRESS

Detailed analysis: The native PTY backend can close the PTY file descriptor while a read goroutine is still active. The goroutine then reads from a closed FD, returning errors or garbage data. Additionally, `Close()` may be called multiple times, causing a double-close.

```
// pkg/session/backends/native.go (simplified)
func (n *NativeBackend) Close() error {
    return n.pty.Close() // may be called while Read
        goroutine is active
}

func (n *NativeBackend) Read(p []byte) (int, error) {
    return n.pty.Read(p) // reads from potentially closed
        PTY
}
```

Remediation: 1. Use `sync.Once` for close 2. Wait for read goroutine to finish before closing 3. Use a dedicated “done” channel to signal read goroutine

F6: Unbounded Scheduler Placements Map

Attribute	Value
Location	`pkg/scheduler/scheduler.go` — placements map
Class	Memory leak (unbounded map)

Attribute	Value
Severity	Major
Confidence	High
Status	IN PROGRESS

Detailed analysis: The scheduler maintains a `placements` map that tracks scheduling decisions. Entries are added but never removed (no completion path), causing unbounded memory growth.

```
// pkg/scheduler/scheduler.go (simplified)
type Scheduler struct {
    placements map[string]*Placement // grows forever!
    // ...
}

func (s *Scheduler) Schedule(job *Job) {
    s.placements[job.ID] = &Placement{...} // added
    // ... but never removed when job completes
}
```

Remediation: 1. Remove placements when jobs complete or fail 2. Add periodic cleanup of stale placements 3. Cap map size with eviction policy

F7: Unbounded Goroutines in EventBus

Attribute	Value
Location	`EventBus/pkg/bus/bus.go` — PublishAsync()
Class	Unbounded goroutine spawn
Severity	Major
Confidence	High
Status	IN PROGRESS

Detailed analysis: `PublishAsync()` launches a new goroutine for each event published. Under high load, this can spawn millions of goroutines, exhausting memory and scheduler resources.

```
// EventBus/pkg/bus/bus.go (simplified)
func (b *Bus) PublishAsync(topic string, data interface{}) {
    go func() { // unbounded goroutine spawn!
        b.handlers[topic](data)
```

```
    }()
}
```

Remediation: 1. Use bounded worker pool (semaphore or channel-based) 2. Backpressure mechanism when pool is full 3. Rate limiting on publish

F8: Unbuffered Delivery Stall in NATS

Attribute	Value
Location	`EventBus/pkg/nats/nats.go` — Subscribe()
Class	Unbuffered channel stall
Severity	Minor
Confidence	Medium
Status	IN PROGRESS

Detailed analysis: The NATS subscriber uses unbuffered channels for message delivery. If the consumer is slow, the NATS connection blocks, potentially causing backpressure and connection drops.

Remediation: 1. Use buffered channels 2. Implement non-blocking send with drop policy 3. Add flow control

F9: No Size Cap on TieredCache Hot Tier

Attribute	Value
Location	`pkg/tieredcache/tieredcache.go` — Hot tier
Class	Unbounded cache growth
Severity	Minor
Confidence	High
Status	QUEUED (next wave)

Detailed analysis: The hot tier of the tiered cache has no size cap. The maintenance loop only evicts expired entries but doesn’t enforce a maximum size. Under sustained writes, the cache can grow without bound.

Remediation: 1. Add LRU eviction with configurable max size 2. Periodic size-based eviction 3. Memory pressure monitoring

F10: Per-Event Timer Allocation on Hot Path

Attribute	Value
Location	`EventBus/pkg/bus/bus.go` — trySend()
Class	Performance hazard (GC pressure)
Severity	Minor
Confidence	Medium
Status	IN PROGRESS

Detailed analysis: `trySend()` allocates a `time.NewTimer()` for every event send attempt. Under high throughput, this creates massive GC pressure.

```
// EventBus/pkg/bus/bus.go (simplified)
func (b *Bus) trySend(ch chan interface{}, data
        interface{}) bool {
    timer := time.NewTimer(100 * time.Millisecond) //
        allocation per event!
    defer timer.Stop()
    select {
    case ch <- data:
        return true
    case <-timer.C:
        return false
    }
}
```

Remediation: 1. Use `time.AfterFunc` with a shared timer 2. Or use `select` with default case (non-blocking send) 3. Pool timer objects

4.2 Thread-Safety Analysis Summary

Package	Thread-Safe?	Mechanism	Issues
`pkg/swim`	⚠️ Partial	Mutex on members, but State accessed without lock	F1, F2, F3, F4
`pkg/session`	⚠️ Partial	CRDT merge is safe, PTY backend is not	F5

Package	Thread-Safe?	Mechanism	Issues
`pkg/scheduler`	⚠️ Partial	Mutex on queue, but placements map unguarded	F6
`EventBus`	❌ No	No bounds on goroutine spawning	F7, F8, F10
`pkg/tieredcache`	⚠️ Partial	RWMutex on cache, but no size cap	F9
`pkg/discovery`	✅ Yes	Mutex-protected registry	None
`pkg/events`	✅ Yes	Channel-based concurrency	None
`pkg/resources`	✅ Yes	Read-only after init	None
`pkg/security`	✅ Yes	Mutex on TLS state	None
`pkg/wireguard`	✅ Yes	wgctrl client is thread-safe	None

Chapter 5: Security Assessment

5.1 SPIFFE/SPIRE Implementation

Implementation location: `internal/security/spiffe_ca.go`, `pkg/security/spiffe.go`, `pkg/spiffefed/spiffefed.go`

Status: Partially implemented

Details: - SPIFFE CA implemented in `internal/security/spiffe ca.go` with X.509 SVID issuance - SPIFFE client in `pkg/security/spiffe.go` for workload API interaction - SPIFFE federation in `pkg/spiffefed/spiffefed.go` for cross-cluster trust - Integration tests exist (`internal/security/spiffe mtls integration test.go`) - Unit tests for CA operations (`internal/security/spiffe_ca_unit_test.go`)

Gaps: - Not all services use SPIFFE identity - Federation trust store not populated - Workload API registrar not implemented - mTLS e2e incomplete (HXC-600)

5.2 TLS/Certificate Management

Implementation location: `pkg/security/tls.go`, `internal/gateway/auth.go`

Status: Partially implemented

Details: - TLS configuration generation in `pkg/security/tls.go` - TLS evidence tests in `pkg/security/tls_evidence_test.go` - Gateway HTTPS support on :8443 - Certificate rotation not automated

Gaps: - No automated certificate rotation - No certificate pinning - TLS configuration not enforced for inter-service communication

5.3 JWT/RBAC Enforcement

Implementation location: `internal/gateway/auth.go`, `internal/security/scopes.go`, `internal/security/identity_bindings.go`

Status: CRITICAL GAP

Details: - JWT middleware exists in gateway but accepts stub tokens (CRIT-1) - RBAC scopes defined in `internal/security/scopes.go` - Identity bindings in `internal/security/identity_bindings.go` - Unit tests for RBAC scopes (`internal/security/rbac_scopes_unit_test.go`) - Integration tests for RBAC scopes (`internal/security/rbac_scopes_integration_test.go`) - Authorization tests (`internal/security/authorize_rbac_test.go`)

Gaps: - **CRITICAL**: JWT tokens not verified — accepts any token - No token expiration enforcement - No refresh token mechanism - RBAC scopes not enforced at gateway level - No role hierarchy - Temporal keyset rotation not implemented (partially in `pkg/jwt/temporal_keyset.go`)

5.4 Post-Quantum Key Exchange

Implementation location: `pkg/hybridkex/hybridkex.go`, `pkg/x25519session/session.go`

Status: Implemented with testing

Details: - Hybrid key exchange combining ML-KEM-768 (Kyber) with X25519 - Fuzz testing in `pkg/hybridkex/fuzz_test.go` - X25519 session key agreement in `pkg/x25519session/session.go` - Fuzz testing for X25519 in `pkg/x25519session/fuzz_test.go` - Double encryption envelope in `pkg/doublecrypt/doublecrypt.go`

Gaps: - No NIST compliance certification - No side-channel resistance testing - Not wired into E2EE proxy production path - Key rotation not automated

5.5 Vault Integration

Implementation location: `pkg/security/vault.go`, `pkg/security/vault_api_client.go`, `pkg/security/secret_injector.go`

Status: Implemented with testing

Details: - Vault client in `pkg/security/vault api client.go` using `hashicorp/vault/api` - Secret injection in `pkg/security/secret injector.go` - Integration test in `pkg/security/secret injector integration test.go` - Vault API client test in `pkg/security/vault api client test.go` - Vault integration test in `pkg/security/vault_api_client_integration_test.go`

Gaps: - Requires running Vault server for integration tests - Secret injector not wired into all services - No dynamic secret rotation - No Vault policy management

5.6 Known Vulnerabilities

ID	Severity	Description	Status
SEC-1	CRITICAL	JWT tokens not verified — accepts any token	Open
SEC-2	HIGH	No mTLS between services	Partial (HXC-600)
SEC-3	HIGH	No secret rotation	Open
SEC-4	MEDIUM	govulncheck: 0 reachable vulns (clean)	Verified
SEC-5	MEDIUM	No Snyk/ SonarQube scanning	Open (needs tokens)
SEC-6	MEDIUM	gosec not promoted to gate	Open
SEC-7	LOW	No container image scanning (trivy)	Partial
SEC-8	LOW		

ID	Severity	Description	Status
		No SBOM in release artifacts	Partial (cyclonedx-gomod wired)

5.7 Remediation Priority

1. **P0**: Fix JWT token verification (CRIT-1)
2. **P0**: Enable mTLS between all services
3. **P1**: Promote gosec to build gate
4. **P1**: Wire secret injector into all services
5. **P2**: Implement secret rotation
6. **P2**: Add Snyk/SonarQube (requires tokens)
7. **P3**: Container image scanning
8. **P3**: SBOM in release artifacts

Chapter 6: Database Schema Analysis

6.1 PostgreSQL Migrations (001–015)

Migration 001: Create Nodes

```
 -- migrations/postgresql/001_create_nodes.up.sql
CREATE TABLE nodes (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    hostname TEXT NOT NULL,
    ip_address INET NOT NULL,
    port INTEGER NOT NULL DEFAULT 50053,
    status TEXT NOT NULL DEFAULT 'unknown',
    labels JSONB DEFAULT '{}',
    gpu_count INTEGER DEFAULT 0,
    gpu_type TEXT,
    total_memory BIGINT DEFAULT 0,
    total_cpu_cores INTEGER DEFAULT 0,
    last_heartbeat TIMESTAMPTZ,
    registered_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

Migration 002: Create GPU Devices

```
-- migrations/postgresql/002_create_gpu_devices.up.sql
CREATE TABLE gpu_devices (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    node_id UUID NOT NULL REFERENCES nodes(id) ON DELETE
        CASCADE,
    gpu_index INTEGER NOT NULL,
    gpu_uuid TEXT,
    gpu_name TEXT NOT NULL,
    memory_total BIGINT NOT NULL,
    memory_used BIGINT DEFAULT 0,
    utilization REAL DEFAULT 0,
    temperature REAL DEFAULT 0,
    power_usage REAL DEFAULT 0,
    mig_enabled BOOLEAN DEFAULT FALSE,
    mig_profiles JSONB DEFAULT '[]',
    status TEXT NOT NULL DEFAULT 'available',
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    UNIQUE(node_id, gpu_index)
);
```

Migration 003: Create Sessions

```
-- migrations/postgresql/003_create_sessions.up.sql
CREATE TABLE sessions (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id TEXT NOT NULL,
    name TEXT NOT NULL,
    backend TEXT NOT NULL DEFAULT 'native',
    status TEXT NOT NULL DEFAULT 'creating',
    node_id UUID REFERENCES nodes(id),
    working_dir TEXT,
    command TEXT,
    environment JSONB DEFAULT '{}',
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    closed_at TIMESTAMPTZ
);
```

Migration 004: Create Session Windows

```
CREATE TABLE session_windows (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

```
    session_id UUID NOT NULL REFERENCES sessions(id) ON
        DELETE CASCADE,
    window_index INTEGER NOT NULL,
    title TEXT,
    active BOOLEAN DEFAULT FALSE,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    UNIQUE(session_id, window_index)
);
```

Migration 005: Create Session Panes

```
CREATE TABLE session_panes (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    window_id UUID NOT NULL REFERENCES session_windows(id)
        ON DELETE CASCADE,
    pane_index INTEGER NOT NULL,
    current_command TEXT,
    working_dir TEXT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

Migration 006: Create Reservations

```
CREATE TABLE reservations (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    node_id UUID NOT NULL REFERENCES nodes(id),
    gpu_device_id UUID REFERENCES gpu_devices(id),
    job_id UUID,
    resource_type TEXT NOT NULL,
    resource_amount INTEGER NOT NULL,
    status TEXT NOT NULL DEFAULT 'active',
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    released_at TIMESTAMPTZ
);
```

Migration 007: Create Scheduling Queue

```
CREATE TABLE scheduling_queue (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    job_name TEXT NOT NULL,
    user_id TEXT NOT NULL,
    priority INTEGER NOT NULL DEFAULT 0,
    resource_requirements JSONB NOT NULL,
    status TEXT NOT NULL DEFAULT 'queued',
    assigned_node_id UUID REFERENCES nodes(id),
```

```
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    started_at TIMESTAMPTZ,
    completed_at TIMESTAMPTZ,
    error_message TEXT
);
```

Migration 008: Create Health Snapshots

```
CREATE TABLE health_snapshots (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    node_id UUID NOT NULL REFERENCES nodes(id),
    check_type TEXT NOT NULL,
    status TEXT NOT NULL,
    message TEXT,
    latency_ms REAL,
    checked_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

Migration 009: Create LLM Advisories

```
CREATE TABLE llm_advisories (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    model_id TEXT NOT NULL,
    advisory_type TEXT NOT NULL,
    content TEXT NOT NULL,
    confidence REAL DEFAULT 0,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

Migration 010: Create Audit Log

```
CREATE TABLE audit_log (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id TEXT NOT NULL,
    action TEXT NOT NULL,
    resource_type TEXT NOT NULL,
    resource_id TEXT,
    details JSONB DEFAULT '{}',
    ip_address INET,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

Migration 011: Create Build Jobs

```
CREATE TABLE build_jobs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id TEXT NOT NULL,
    source_url TEXT NOT NULL,
    build_type TEXT NOT NULL DEFAULT 'go',
    status TEXT NOT NULL DEFAULT 'queued',
    config JSONB DEFAULT '{}',
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    started_at TIMESTAMPTZ,
    completed_at TIMESTAMPTZ,
    error_message TEXT
);
```

Migration 012: Create Build Artifacts

```
CREATE TABLE build_artifacts (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    build_job_id UUID NOT NULL REFERENCES build_jobs(id) ON
        DELETE CASCADE,
    artifact_type TEXT NOT NULL,
    path TEXT NOT NULL,
    checksum TEXT NOT NULL,
    size BIGINT NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

Migration 013: Create Users

```
CREATE TABLE users (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    username TEXT NOT NULL UNIQUE,
    email TEXT NOT NULL UNIQUE,
    password_hash TEXT NOT NULL,
    roles JSONB DEFAULT '[]',
    status TEXT NOT NULL DEFAULT 'active',
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

Migration 014: Create Migration History

```
CREATE TABLE migration_history (
    id SERIAL PRIMARY KEY,
```

```
    version INTEGER NOT NULL UNIQUE,
    name TEXT NOT NULL,
    applied_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

Migration 015: Triggers and Functions

```
-- Auto-update updated_at on row modification
CREATE OR REPLACE FUNCTION update_updated_at()
RETURNS TRIGGER AS $$
BEGIN
    NEW.updated_at = NOW();
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- Apply to all tables with updated_at
CREATE TRIGGER update_nodes_updated_at
    BEFORE UPDATE ON nodes
    FOR EACH ROW EXECUTE FUNCTION update_updated_at();

-- Similar triggers for sessions, gpu_devices, users, etc.
```

6.2 Schema Drift Issues

HXC-1639: The `migrations/postgresql/0001_primary_schema.sql` diverges from the numbered migration chain `001-015`. This creates two conflicting sources of truth for the database schema. The drift guard test in `internal/schema/drift_guard_test.go` exists but was not preventing the divergence.

Remediation status: IN PROGRESS — reconcile to single source of truth (canonical = migrate chain) + drift-guard test.

6.3 etcd Key Patterns

Defined in `pkg/etcd/keys.go`:

```
/helix/cluster/{cluster-id}                    — Cluster
metadata
/helix/nodes/{node-id}                         — Node
registration
/helix/nodes/{node-id}/gpus/{gpu-index}        — GPU device
state
/helix/sessions/{session-id}                   — Session
metadata
/helix/leader/{service-name}                   — Leader
```

```
election
/helix/config/{key}                                  — Dynamic
configuration
/helix/discovery/{service-name}/{instance-id}  — Service
discovery
/helix/health/{node-id}                              — Health
state
```

6.4 Redis Usage Patterns

Key Pattern	Type	Purpose	TTL
`ratelimit:{user_id}`	String (counter)	Rate limiting	Configurable
`cache:hot:{key}`	String	Hot tier cache	Configurable
`cache:warm:{key}`	String	Warm tier cache	Longer TTL
`session:{session_id}`	Hash	Session state cache	30 min
`health:{node_id}`	Hash	Health check cache	60 sec
`metrics:{metric_name}`	Sorted Set	Time-series metrics	24 hours

6.5 SQLite Registry Schema

The HXC registry uses SQLite with the following schema (from `pkg/hxcregistry/schema.sql`):

```sql
CREATE TABLE items (
    id TEXT PRIMARY KEY,
    title TEXT NOT NULL,
    status TEXT NOT NULL DEFAULT 'queued',
    priority INTEGER DEFAULT 0,
    phase TEXT,
    package_path TEXT,
    description TEXT,
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE artifacts (
```

```
    id TEXT PRIMARY KEY,
    item_id TEXT NOT NULL REFERENCES items(id),
    artifact_type TEXT NOT NULL,
    path TEXT NOT NULL,
    checksum TEXT,
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

Chapter 7: API Surface Analysis

7.1 gRPC Services (10 Proto Definitions)

`api/v1/scheduler.proto`

```
service SchedulerService {
    rpc Schedule(ScheduleRequest) returns
        (ScheduleResponse);
    rpc Preempt(PreemptRequest) returns (PreemptResponse);
    rpc ListJobs(ListJobsRequest) returns
        (ListJobsResponse);
    rpc CancelJob(CancelJobRequest) returns
        (CancelJobResponse);
    rpc WatchJobs(WatchJobsRequest) returns (stream
        JobEvent);
}
```

`api/v1/session.proto`

```
service SessionService {
    rpc Create(CreateSessionRequest) returns
        (CreateSessionResponse);
    rpc Attach(AttachSessionRequest) returns (stream
        SessionOutput);
    rpc Detach(DetachSessionRequest) returns
        (DetachSessionResponse);
    rpc Resize(ResizeSessionRequest) returns
        (ResizeSessionResponse);
    rpc Kill(KillSessionRequest) returns
        (KillSessionResponse);
    rpc List(ListSessionsRequest) returns
        (ListSessionsResponse);
}
```

api/v1/node.proto

```
service NodeService {
    rpc Register(RegisterNodeRequest) returns
        (RegisterNodeResponse);
    rpc Deregister(DeregisterNodeRequest) returns
        (DeregisterNodeResponse);
    rpc Heartbeat(HeartbeatRequest) returns
        (HeartbeatResponse);
    rpc ListNodes(ListNodesRequest) returns
        (ListNodesResponse);
    rpc GetNode(GetNodeRequest) returns (GetNodeResponse);
}
```

api/v1/health.proto

```
service HealthService {
    rpc Check(HealthCheckRequest) returns
        (HealthCheckResponse);
    rpc Watch(HealthWatchRequest) returns (stream
        HealthCheckResponse);
    rpc Aggregate(AggregateRequest) returns
        (AggregateResponse);
}
```

api/v1/security.proto

```
service SecurityService {
    rpc Authenticate(AuthenticateRequest) returns
        (AuthenticateResponse);
    rpc Authorize(AuthorizeRequest) returns
        (AuthorizeResponse);
    rpc IssueToken(IssueTokenRequest) returns
        (IssueTokenResponse);
    rpc ValidateToken(ValidateTokenRequest) returns
        (ValidateTokenResponse);
}
```

api/v1/build.proto

```
service BuildService {
    rpc Submit(SubmitBuildRequest) returns
        (SubmitBuildResponse);
    rpc Status(BuildStatusRequest) returns
        (BuildStatusResponse);
```

```
    rpc Cancel(CancelBuildRequest) returns
        (CancelBuildResponse);
    rpc Logs(BuildLogsRequest) returns (stream
        BuildLogEntry);
    rpc List(ListBuildsRequest) returns
        (ListBuildsResponse);
}
```

`api/v1/advisory.proto`

```
service AdvisoryService {
    rpc GetAdvice(GetAdviceRequest) returns
        (GetAdviceResponse);
    rpc ListAdvisories(ListAdvisoriesRequest) returns
        (ListAdvisoriesResponse);
}
```

`api/v1/gpu_proxy.proto`

```
service GPUProxyService {
    rpc Allocate(AllocateGPURequest) returns
        (AllocateGPUResponse);
    rpc Release(ReleaseGPURequest) returns
        (ReleaseGPUResponse);
    rpc Status(GPUStatusRequest) returns
        (GPUStatusResponse);
}
```

`api/v1/edge.proto`

```
service EdgeService {
    rpc Register(RegisterEdgeRequest) returns
        (RegisterEdgeResponse);
    rpc Heartbeat(EdgeHeartbeatRequest) returns
        (EdgeHeartbeatResponse);
    rpc ScheduleWork(ScheduleWorkRequest) returns
        (ScheduleWorkResponse);
}
```

`api/v1/types.proto`

Shared message types used across services: `Node`, `GPU`, `Job`, `Session`, `ResourceRequirements`, etc.

7.2 REST Endpoints (Gateway API)

The gateway exposes the following REST API documented in `internal/gateway/openapi.yaml`:

Cluster Overview

Method	Path	Description
GET	`/api/v1/cluster/status`	Cluster status overview
GET	`/api/v1/cluster/metrics`	Cluster-level metrics

Nodes

Method	Path	Description
GET	`/api/v1/nodes`	List all nodes
GET	`/api/v1/nodes/{id}`	Get node details
POST	`/api/v1/nodes`	Register new node
DELETE	`/api/v1/nodes/{id}`	Deregister node
GET	`/api/v1/nodes/{id}/gpus`	List node GPUs

Sessions

Method	Path	Description
GET	`/api/v1/sessions`	List sessions
POST	`/api/v1/sessions`	Create session
GET	`/api/v1/sessions/{id}`	Get session details
DELETE	`/api/v1/sessions/{id}`	Kill session
POST	`/api/v1/sessions/{id}/resize`	Resize session

Jobs

Method	Path	Description
GET	`/api/v1/jobs`	List jobs
POST	`/api/v1/jobs`	Submit job

Method	Path	Description
GET	`/api/v1/jobs/{id}`	Get job status
DELETE	`/api/v1/jobs/{id}`	Cancel job

Builds

Method	Path	Description
GET	`/api/v1/builds`	List builds
POST	`/api/v1/builds`	Submit build
GET	`/api/v1/builds/{id}`	Get build status
DELETE	`/api/v1/builds/{id}`	Cancel build
GET	`/api/v1/builds/{id}/logs`	Get build logs

Health

Method	Path	Description
GET	`/api/v1/health`	Health aggregation
GET	`/api/v1/health/{node}`	Node-specific health

Security

Method	Path	Description
POST	`/api/v1/security/authenticate`	Authenticate user
POST	`/api/v1/security/authorize`	Check authorization
POST	`/api/v1/security/token`	Issue token

Inference

Method	Path	Description
POST	`/api/v1/inference/completion`	LLM completion
POST	`/api/v1/inference/chat`	LLM chat

7.3 WebSocket Streams

Path	Service	Message Type	Direction
`/ws/terminal/{session}`	HTMUX	Binary (terminal I/O)	Bidirectional
`/ws/session/{id}/attach`	Session	Binary (session output)	Server→Client
`/ws/health/watch`	Health	JSON (health events)	Server→Client
`/ws/build/{id}/logs`	Build	Text (build logs)	Server→Client
`/ws/jobs/watch`	Scheduler	JSON (job events)	Server→Client

7.4 OpenAPI Specification Compliance

The OpenAPI specification is located at `internal/gateway/openapi.yaml`. The `pkg/openapivalidate/openapivalidate.go` package provides validation against the spec.

Compliance status: - ✅ Basic CRUD endpoints documented - ⚠️ WebSocket streams not documented in OpenAPI - ⚠️ Error response schemas incomplete - ⚠️ Authentication headers not fully specified - ❌ Inference proxy endpoints not fully documented - ❌ Rate limiting headers not specified

Remediation: 1. Complete OpenAPI spec with all endpoints 2. Add error response schemas 3. Document authentication requirements 4. Add WebSocket specification (AsyncAPI) 5. Wire `pkg/openapivalidate` into gateway middleware

---

End of Comprehensive Analysis Report

Document Statistics: - Total sections: 7 chapters + executive summary - Total packages analyzed: 212+ - Total binaries analyzed: 24 - Total concurrency hazards: 10 - Total security findings: 8 - Total database migrations: 15 + 1 primary - Total proto definitions: 10 - Total REST endpoints: 25+ - Total WebSocket streams: 5